

Contents

1	DISCLAIMER	5
2	Preface	6
2.1	Code Conversion Complexities	6
2.1.1	Java to Cangjie Conversion Challenges	7
2.2	What J2CJ Is And Is Not	7
3	Installation And Setup	8
3.1	Version Information	8
3.2	System Requirements	8
3.2.1	IDE Plugin	8
3.3	Installation	8
3.3.1	IntelliJ IDEA Plugin Installation	9
3.3.2	IntelliJ IDEA Plugin Configuration	10
4	Usage	13
4.1	IntelliJ IDEA Plugin Usage	13
4.2	J2CJ Command Line	19
4.2.1	Command-Line Options	20
4.3	Advanced Usage	22
4.3.1	Incremental Translation	24
4.3.2	Semantics Preservation	25
4.3.2.1	Types of volatile Fields	26
4.3.2.2	Function Types	28
4.4	Translation Result	29
5	Mappings	30
5.1	Mappings Loading Process	30
5.2	Adding Custom Mappings	30
5.3	Mapping Syntax	31
5.3.1	Cangjie Declarations	31
5.3.1.1	Cangjie Declaration Examples	32
5.3.1.2	Extension Declarations	33
5.3.2	Java Mappings	33
5.3.2.1	Type Mappings	33
5.3.2.2	Member And Constructor Mappings	34
5.4	Additional Method And Constructor Mapping Considerations	35
5.4.1	Mapping Overloaded Methods And Constructors	35
5.4.1.1	Java Signatures	35
5.4.2	Mapping Inherited Methods	36
5.5	More Simple Mapping Examples	37
6	Annotation Support	38
6.1	Predefined Annotation Types	38

6.2	User-Defined Annotation Types	38
6.2.1	Annotation Type Definitions	38
6.2.2	Annotation Usages	39
6.3	Annotation Mappings	39
7	Java Features Not Supported by J2CJ	40
7.1	Lexical Structure	40
7.1.1	Comments	40
7.1.2	Literals	42
7.1.2.1	Numeric Literals	42
7.1.2.2	String Literals	42
7.2	Types, Values, and Variables	42
7.2.1	Reference Types	42
7.2.2	Parameterized Types	43
7.2.3	Intersection Types	43
7.3	Conversions and Contexts	43
7.3.1	Boxing Conversion	43
7.3.2	Unboxing Conversion	44
7.3.3	String Conversion	44
7.4	Names	44
7.4.1	Fully Qualified Names	45
7.4.2	Scoping???	45
7.5	Packages	45
7.5.1	Static Import	45
7.6	Classes	45
7.6.1	Class Declarations	45
7.6.1.1	Class Modifiers	45
7.6.1.1.1	sealed, non-sealed, and final Classes	45
7.6.1.1.2	strictfp Classes	46
7.6.1.2	Inner Classes and Enclosing Instances	46
7.6.2	Field Declarations	46
7.6.2.1	transient Fields	46
7.6.2.2	volatile Fields	46
7.6.3	Method Declarations	46
7.6.3.1	Method Modifiers	46
7.6.3.1.1	native Methods	46
7.6.3.2	synchronized Methods	47
7.6.3.2.1	strictfp Methods	47
7.6.4	Instance Initializers	47
7.6.5	Static Initializers	48
7.6.6	Enum Types	48
7.6.7	Record Classes	48
7.7	Interfaces	48
7.7.1	Interface Declarations	48
7.7.2	Field (Constant) Declarations	48
7.7.3	Method Declarations	49

7.7.4	Annotation Types	50
7.7.4.1	Predefined Annotation Types	50
7.7.5	Functional Interfaces	50
7.8	Blocks and Statements	50
7.8.1	Labeled Statements	50
7.8.2	switch Statements	50
7.8.3	Basic for Statements	50
7.8.4	Enhanced for Statements	52
7.8.5	break Statements	52
7.8.6	continue Statements	52
7.8.7	synchronized Statements	52
7.8.8	try Statements	53
7.8.9	Pattern Matching	53
7.9	Expressions	53
7.9.1	Primary Expressions	53
7.9.1.1	this	53
7.9.1.2	Qualified this	54
7.9.2	Field Access Expressions	54
7.9.3	Method Invocation Expressions	54
7.9.4	Method Reference Expressions	55
7.9.5	Cast Expressions	55
7.9.6	Multiplicative Operators	56
7.9.6.1	Remainder Operator %	56
7.9.7	Shift Operators	56
7.9.8	Relational Operators	56
7.9.8.1	Type Comparison Operator instanceof	56
7.9.9	Lambda Expressions	57
7.10	Miscellaneous Non-Language Issues	58
7.10.1	Exception Chaining	58
7.10.2	Metatype Programming	59
8	Markers	60
8.1	No type mapping: <CLASS_FQ_NAME>	60
8.2	Missing mapping for <CLASS_FQ_NAME> member/constructor: <MEMBER_NAME>	60
8.3	Missing method of mapped type <CLASS_FQ_NAME>: <MEMBER_NAME>	60
8.4	Mapping is present but failed	60
8.5	Unimplemented feature <FEATURE_NAME>	60
8.6	Unsupported <FEATURE_DESCRIPTION>	61
8.7	No type transform for: <EXPRESSION> <FROM_TYPE> -> <TO_TYPE>	61
8.8	Cannot infer type argument: <TYPE_PARAMETER>	61
8.9	Generic instantiation failure: <CALL>	62
8.10	Constraints not satisfied <FAILURES>: <CALL>	62
8.11	Expression is too deep: <EXPRESSION>	62
8.12	Unsupported modifier: <MODIFIER>	62
8.13	Constants in interfaces with bounded type parameters are unsupported: <INITIALIZER>	63
8.14	Unary operators usage with volatile fields is not supported: <EXPRESSION>	64

8.15	Compound assignment to volatile fields is not supported: <EXPRESSION>	64
8.16	Internal error	64
9	Solutions to Common Problems	65
9.1	Version Compatibility	65

1 DISCLAIMER

1. This tool is provided solely for reference purposes. Unless required by applicable laws or otherwise agreed in writing, this tool is provided “as is”. The owner, developer, and provider of this tool do not provide any express or implied warranty and shall not be liable for any direct or indirect loss or damage caused by the use of the tool, including but not limited to loss of profits, data loss, business interruption, etc. The user shall bear any risk, liability and/or any loss arising from the use of the tool.
2. The code generated by this tool may be incomplete, incorrect, or defective. Users need to review and test the code to ensure its applicability and correctness.
3. The code converted by this tool may be protected by intellectual property rights or other legal rights. Users must comply with all applicable laws and regulations and do not infringe any third-party rights. The owner, developer, and provider of this tool shall not bear any direct or indirect liability such as joint liability or tort liability for any infringement of third-party rights by users.
4. The owner, developer, and provider of this tool do not guarantee the quality, marketability, security, or performance of the software developed by users using this tool. Users shall bear related risks and responsibilities.
5. The rights holder of this tool has the right to modify the contents of this disclaimer, and the user shall regularly consult the latest applicable version.

2 Preface

J2CJ is a tool designed to aid Java to Cangjie translation efforts, automating 80-90% of the job.

The **next section** explains why the remaining 10-20% still require manual labor and what precludes that percentage from ever dropping to zero.

2.1 Code Conversion Complexities

Ideally, we want the result of a source to source conversion effort to have the following properties:

- **It works:** Assuming that the original program is correct, the converted program produces the same output when given the same input.
- **It's fast:** Unless the target language is inherently slower, e.g. due to dynamic typing or less mature implementation, the key performance characteristics of the converted program are on par with that of the original program.
- **It's pretty:** The source code of the converted program is readable and maintainable; all comments are preserved and located at the right places.

The first (making it work) is relatively easy to achieve for *self-contained* code if the source and target languages are fairly similar. However, pieces of code that rely on the standard library of the source language may only have a chance to be converted with such ease if the conversion occurs *within an ecosystem*, e.g. from one JVM or CLR language to another, or, better yet, from a legacy version of a language to the current one.

In the more general case, *mappings* have to be defined prior to the conversion between the types and functions defined in the *standard library* of the source language (and possibly also *operating system/foreign function APIs* definitions) and their counterparts in the target language environment. Fortunately, those mappings only need to cover the types and functions that are *actually used* in the source code corpus that is subject to conversion. Good news end here, though, as defining such mappings for third-party libraries and/or frameworks may be too hard, if not outright impossible, and their own code often cannot be converted for technical, legal, or budget reasons.

Performance (making it fast) may be tough to achieve if certain features of the source language have no direct equivalents in the target one and must therefore be emulated. If such features are *frequently used* in performance-critical code, the emulation overheads may be quite substantial. And even if that is not the case, the memory footprint of the program will most likely increase.

With regard to the readability and maintainability of the source code yielded by the converter (making it pretty), those properties are to a large extent linked to the previous item, as language feature simulation code can get rather bulky and ugly.

By way of example, suppose the source language supports exceptions, whereas the target language does not. If the latter is a low-level language, there may be a possibility to simulate exception handling with “non-local **goto**” functionality found in its standard library, such as `setjmp()` / `longjmp()` in C. Problem is, the resulting code would be very hard to read and near impossible to maintain even if the use of exceptions in the original code is fairly modest. That said, the complexity of control structures that would have to be generated in the absence of such low-level feature in the target language may overwhelm its compiler and IDE, let alone the developers tasked with maintenance of such code.

On top of that, special care and analysis may be required for proper preservation and placement of comments, which are not technically part of the program and therefore can be formatted and placed rather arbitrarily.

2.1.1 Java to Cangjie Conversion Challenges

Both Java and Cangjie are general purpose, object-oriented imperative languages that have many similarities at first sight. But a closer look reveals a handful of fundamental differences, and lots of more subtle nuances. To give you a few examples of the former:

- There is no null type in Cangjie. As a result, any use of a reference type in the original Java program cannot be translated into a use of a Cangjie reference type, except when it can be proven that a **null** reference may never emerge. However, the latter requires a more sophisticated converter capable of code flow analysis.
- Cangjie strings and arrays are not objects, whereas in Java they are. In addition, Cangjie arrays are not covariant.
- Type declarations in Cangjie may not be nested. In particular, this means that the semantics of Java inner classes has to be simulated, which requires explicit declaration and initialization of references to enclosing instances, and the relaxation of access modifiers.

As to the standard library, the Java SE API is vast and rich compared to Cangjie, simply because Java is older.

2.2 What J2CJ Is And Is Not

Given the above considerations, making J2CJ a fully automated translator would be, on the one hand, too expensive and on the other hand, pointless, as certain parts of the resulting Cangjie code would be hard to read and maintain and at the same time incur severe performance and memory overheads.

Therefore J2CJ has been designed from the ground up to *aid* the conversion of Java libraries, applications and frameworks to Cangjie, but not completely replace a specialist with a commanding knowledge of both languages. When a substantial difference between the languages precludes automatic translation, J2CJ generates a **marker**, for that specialist to decide what is more effective: postprocess or manually edit the Cangjie code produced by the translator, or change the original Java source code so that it better maps onto Cangjie. See *Advanced Usage* for details.

3 Installation And Setup

3.1 Version Information

J2CJ version: 1.7.2

Target Cangjie version: 1.0.0

3.2 System Requirements

Hardware/OS: Any system capable of compiling and running console Java 17 applications.

Software: JDK 17, e.g. OpenJDK 17 from <https://jdk.java.net/archive/> or Eclipse Temurin.

3.2.1 IDE Plugin

IntelliJ IDEA 2021.3 or above, or

Android Studio Dolphin 2021.3.1 or above

3.3 Installation

1. If you do not have a JDK 17 installed, install it now.

NOTE: This is *not* the JDK used to build and run your Java projects, but a JDK on which the J2CJ translator itself will run. That is why a specific version is required. But if your projects are targeting the same version of the JDK, feel free to use the existing installation.

2. Unpack the J2CJ archive, `j2cj-distrib-1.7.2-1.0.0.zip`, into a desirable file system location, e.g. `/opt/j2cj`.
3. Verify the installation by running the following command in the terminal:

```
/path/to/jdk/17/bin/java -jar /path/to/j2cj/j2cj.jar
```

For example:

```
/opt/openjdk-17.0.2/bin/java -jar /opt/j2cj/j2cj.jar
```

If the following lines start to appear:

```
Loading built-in root.cjmap
Loading built-in stdlib.cjmap
Loading built-in mappings.cjmap
```

followed by J2CJ usage instructions, you now have everything you need to begin using J2CJ from the command line.

4. If you want to use J2CJ from an IDE such as IntelliJ IDEA or Android Studio, refer to the following subsections for further instructions.

3.3.1 IntelliJ IDEA Plugin Installation

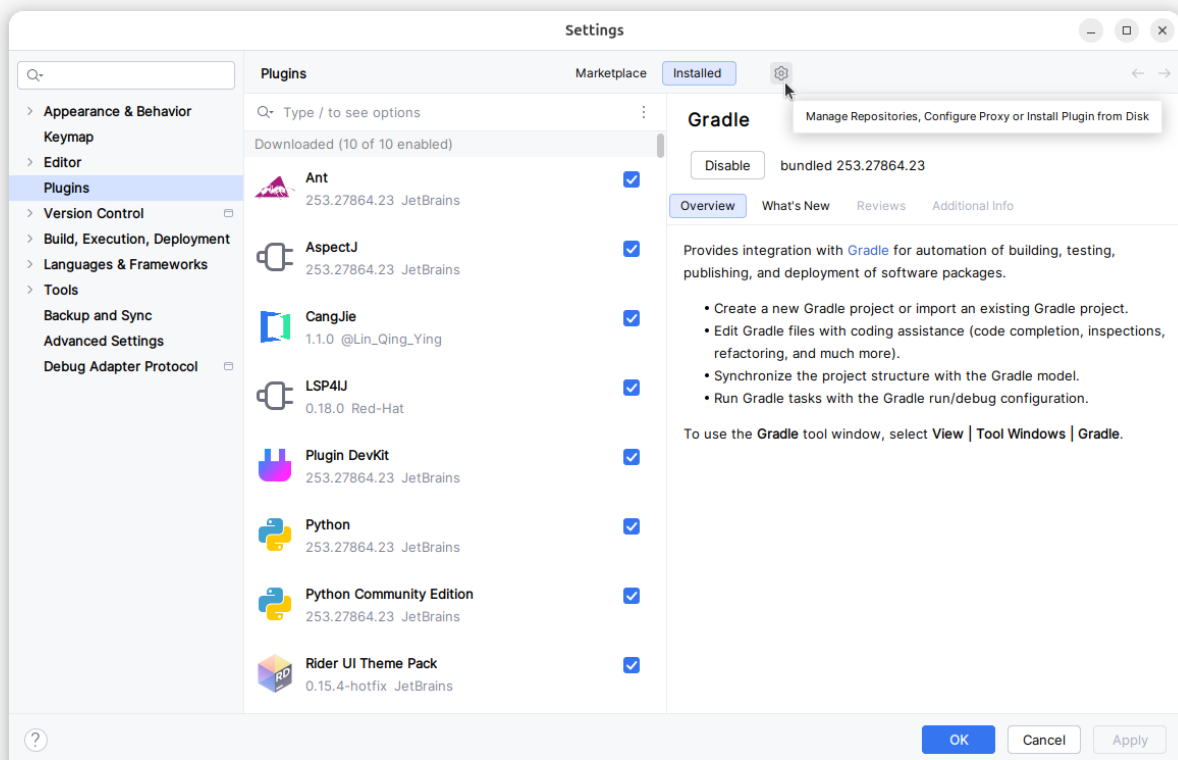
IMPORTANT: If you already have an older version of the plugin installed, make sure to invalidate IntelliJ IDEA/Android Studio caches at the conclusion of the plugin update process, as described below.

NOTE: The below screenshots were taken when running a recent version of IntelliJ IDEA. The respective dialogs may look a bit different in older versions and in Android Studio.

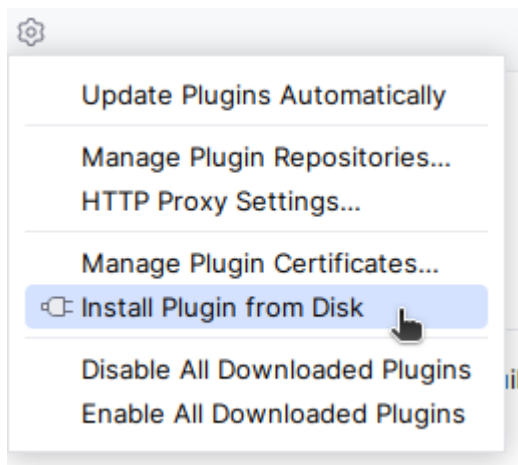
The IntelliJ IDEA plugin for J2CJ supports both Java and Android projects.

To install the plugin, follow these steps:

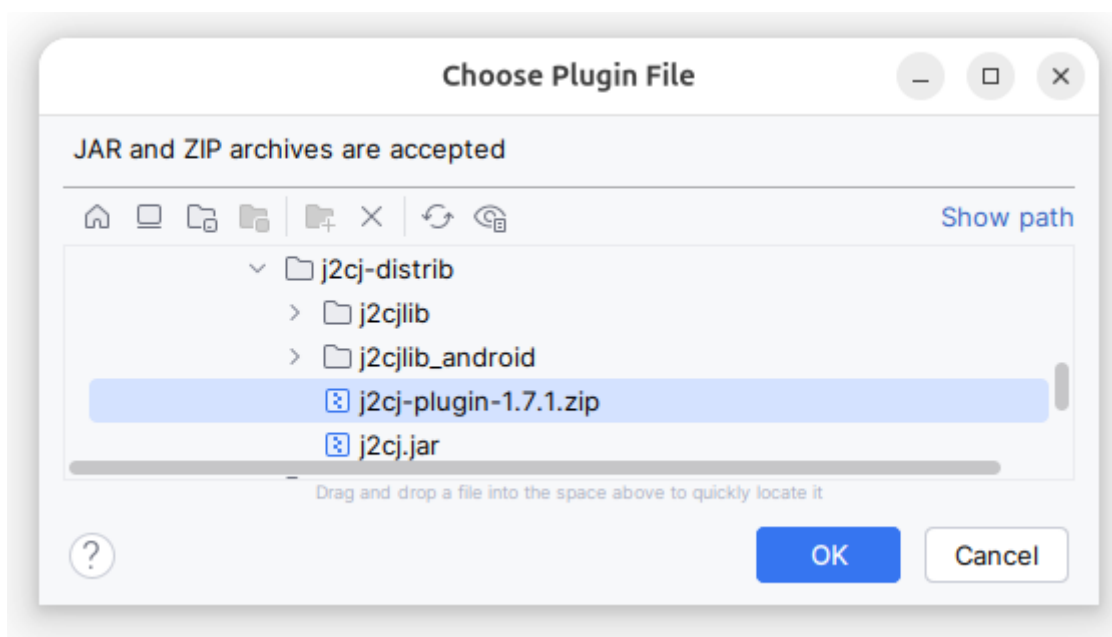
1. Locate the plugin `.zip` file (`j2cj-plugin-1.7.2.zip`) in the directory to which you unpacked J2CJ during **installation**.
2. In the IDE, select **File** -> **Settings** from the main menu, or press **Ctrl+Alt+S** (PC) / **⌘Cmd**, (Mac).
3. In the left column of the **Settings** dialog, select **Plugins**.
4. Click the gear icon just below the window title:



5. Select **Install plugin from disk**.



6. Paste the pathname of the plugin `.zip` file or point to it in the file chooser, then click **OK**.

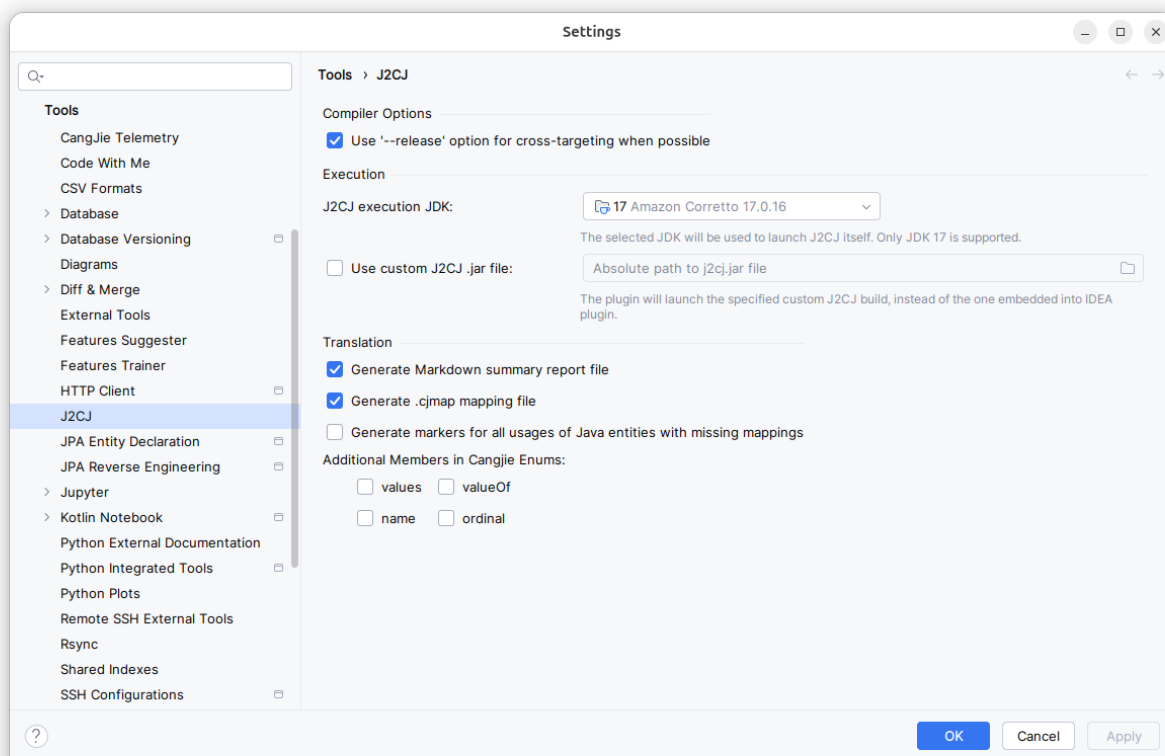


7. **IMPORTANT:** If you have replaced an older plugin version, invalidate the IDE caches **NOW** to ensure proper operation:
 - In the main menu, select **File / Invalidate Caches...**
 - Check the first two boxes: **Clear file system cache and Local History** and **Clear VCS Log caches and indexes**.
 - Click **Invalidate and Restart**.

3.3.2 IntelliJ IDEA Plugin Configuration

To review and/or change the settings of the IntelliJ IDEA Plugin for J2CJ:

1. Select **File -> Settings** from the main menu, or press **Ctrl+Alt+S**. The Settings pop-up window shall open.
2. In the left pane, select **Tools**, then click the **J2CJ** link in the right pane. Alternatively, double-click **Tools** in the left pane, then select **J2CJ** in the list that dropped down. The J2CJ plugin settings tab shall appear:



- Review and/or change the settings, and then click **OK** to confirm your changes, if any, or **Cancel** to close the **Settings** window without saving changes.

The current version of the plugin supports the following basic settings:

- **J2CJ execution JDK** - the JDK used to run J2CJ. The version of the JDK must match the one specified in *System Requirements* which also appears right in this control and is dubbed in the message underneath.

NOTE: This is *not* the JDK used to build and run your Java projects, but a JDK used solely to run the J2CJ translator. That is why a specific version is required. But if your projects are targeting the same version of Java as the one required to run J2CJ, feel free to specify the same JDK.

You would normally change the other settings only under the guidance of a J2CJ support specialist:

- **Use ‘--release’ option for cross-targeting when possible** - normally checked to enforce the use of the `--release` option introduced in the Java 9 `javac` compiler. The plugin is designed to automatically detect situations when the use of that option is mandated; unchecking this checkbox is only ever required for troubleshooting a possible failure of that heuristic.
- **Use custom J2CJ .jar file** - paste the pathname of a standalone J2CJ jar file to be used instead of the one included in the plugin, or point to it in the file chooser, then click **OK**.
- **Generate Markdown summary report file** - normally checked to enforce the generation of a translation report file in Markdown format.
- **Generate .cmap mapping file** - normally checked to make J2CJ save the mappings for the translated classes in a `.cmap` file in the output directory, to enable their use in an *incremental translation* scenario later.

- **Generate markers for all usages of Java entities with missing mappings** - normally unchecked; checking may help with the development of custom **mappings**, but the translation ratio figures will be distorted, as *each* use of a *single* missing mapping would get wrapped in a **marker**, thus reducing the *overall* translation ratio.
- **Additional Members in Cangjie Enums** - see the description of the `--enum-functions` **command line option**.

4 Usage

The easiest way to use J2CJ is via the **IntelliJ IDEA plugin**, which also works in Android Studio. If for any reason you cannot use either IDE, or want to employ **incremental translation** that the plugin does not support yet, the **J2CJ command line interface** is at your service.

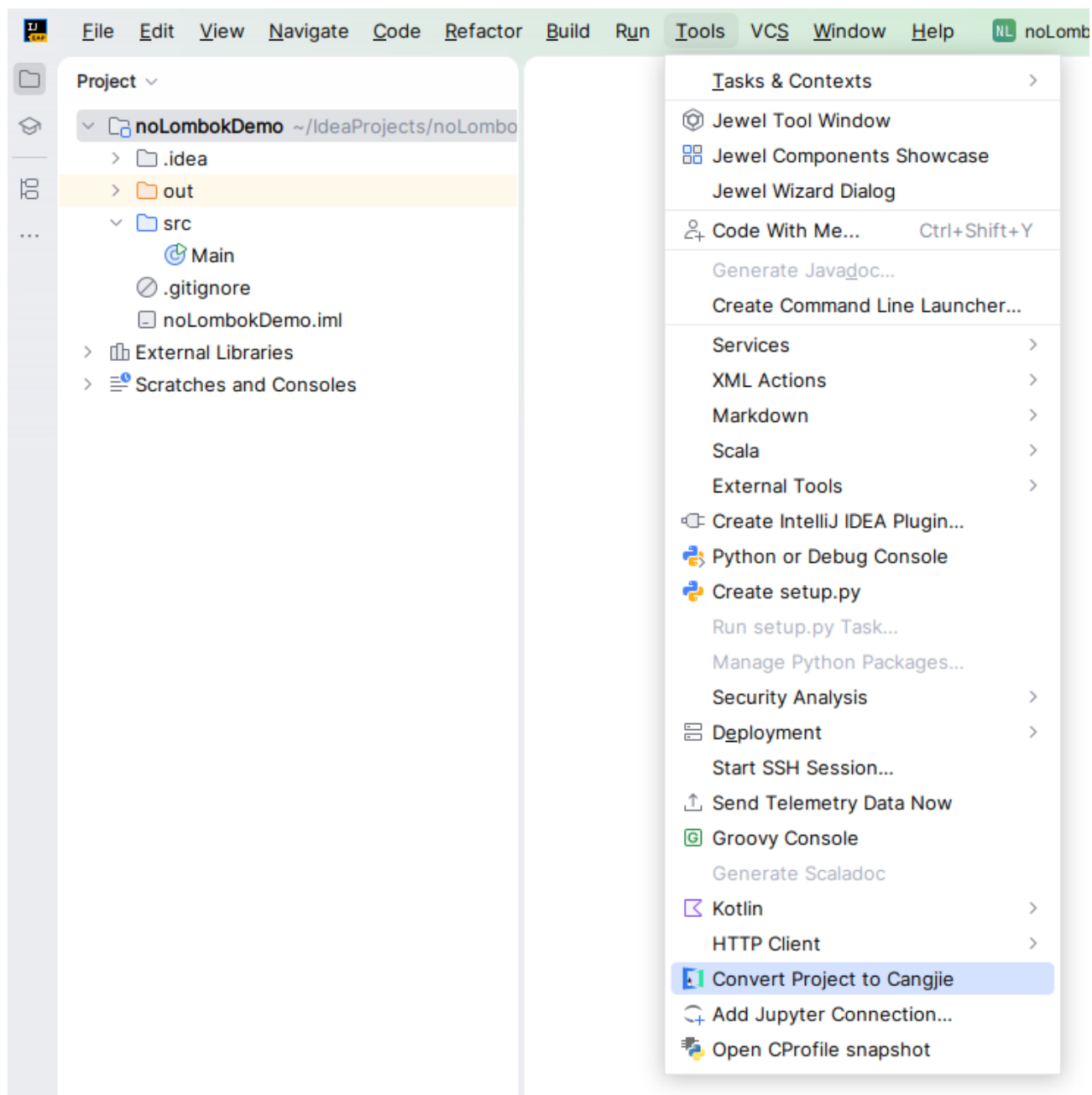
4.1 IntelliJ IDEA Plugin Usage

Refer to the respective sections for plugin **installation** and **configuration** instructions.

IMPORTANT: The J2CJ plugin retrieves crucial information about your Java project from the IDE. However, for projects that rely on third-party build tools such as Apache Maven or Gradle that information may be incorrect, incomplete, or unavailable if you have never built the project in the IDE. Furthermore, J2CJ is unable to translate a project that does not compile, whether due to bugs in the source code or its incompleteness/inconsistency. It is therefore highly recommended to ensure that the project can be successfully built in the IDE before proceeding with the translation.

To translate the source code of your Java/Android project to Cangjie, first complete the following preliminary steps:

1. Open the project in the IDE.
2. Select **Tools -> Convert Project to Cangjie** from the main menu:



A new editor tab shall open:

J2CJ ×

J2CJ execution JDK: <No Project SDK>

Result location: Parent directory for output module directory

Translation mode: ☒ Code style first ? ☐ Preserve semantics ?

Translation scope: ☒ Whole project ☐ Modules

Module name: my_demo

Module version: 1.0.0

☐ Include test sources ?

> Additional Options

Build Project Check Convert Create translation plan

- If the **J2CJ execution JDK** dropdown is displayed, select a JDK 17 installation for J2CJ to run on.

This dropdown will only appear the first time you convert a project and only if you have not pointed the plugin to a JDK installation suitable for running J2CJ when **configuring** it.

J2CJ ×

J2CJ execution JDK: <No Project SDK>

Result location: Parent directory for output module directory

Translation mode: ☒ Code style first ? ☐ Preserve semantics ?

Translation scope: ☒ Whole project ☐ Modules

Module name: my_demo

Module version: 1.0.0

☐ Include test sources ?

> Additional Options

Build Project Check Convert Create translation plan

Detected SDKs

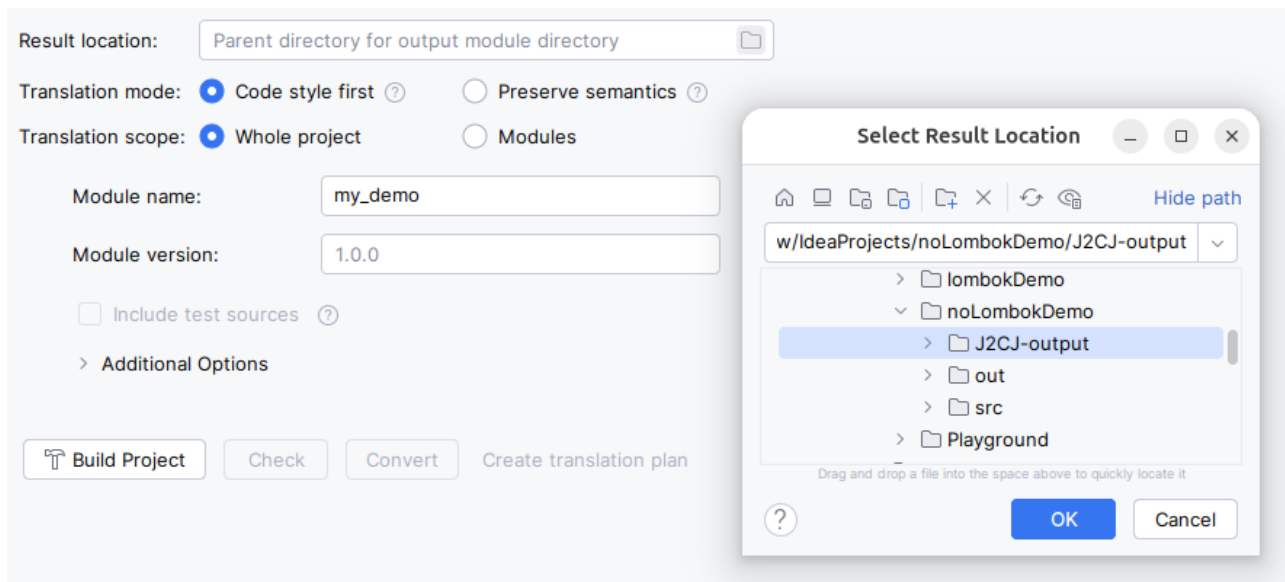
- Java 17.0.16 /usr/lib/jvm/java-1.17.0-openjdk-amd64
- Java 17.0.16 /usr/lib/jvm/java-17-openjdk-amd64
- Amazon Corretto 17.0.15 ~/.jdk/corretto-17.0.15
- Amazon Corretto 17.0.13 ~/.jdk/corretto-17.0.13
- Amazon Corretto 17.0.12 ~/.jdk/corretto-17.0.12
- JetBrains Runtime 17.0.12 ~/.jdk/jbr-17.0.12

+ Add SDK

Download JDK...

- JDK...
- Python SDK...
- IntelliJ Platform Plugin SDK...

- In the **Result location** field, type or paste the pathname of the directory into which J2CJ shall place the results of translation, either absolute or relative to project root, or select the desired directory using a file chooser dialog.



5. Choose the desired **Translation mode**:

- Select **Code style first** (default) to make J2CJ generate more readable and compact Cangjie sources, knowing that they may require in-depth post-translation analysis and editing, or
- Select **Preserve semantics** to preserve as much of the original Java program semantics as possible even if that would increase the verbosity of output Cangjie code and hence the amount of tedious post-translation cleanup work.

Refer to the [Semantics Preservation](#) section for details.

Now, to translate the source code of your *entire* Java/Android project to Cangjie, placing the result in a *single* output Cangjie module, complete the following steps:

6. Optionally, edit the name and version of the output Cangjie module respectively in the **Module name** and **Module version** fields. By default, the IDEA project name is used as the Cangjie module name, with any characters not permitted by CJPM removed. The version number is set to “1.0.0”.
7. Optionally, if the **Include test sources** checkbox is enabled, check it to also translate the source code files that the IDE has recognized as tests.
8. It is highly recommended to conduct a normal build of the project before attempting translation. If you have not built the project recently (or perhaps never), click **Build Project** to initiate the build process, then wait for it to complete.
9. Click **Check**. The plugin shall conduct a dry-run without generating any output files. If the check has detected any problems, wait 30-60 seconds to let the IDE finish updating its internal indices, and repeat the check attempt. **NOTICE:** If your project uses Gradle, you may also use the “Gradle project sync” feature in the IDE to force/speedup the update.
10. Once the check succeeds, click **Convert** to begin the translation process.

J2CJ messages will be displayed in the **Run** pane. If the **Generate Markdown summary report file** box is checked in [plugin settings](#), J2CJ will also generate a report containing translation statistics and diagnostic messages summary in Markdown format. The base name of the report file is the same as the output Cangjie module name specified in **Module name**, and its file name extension is **.md**. The report file is placed in the directory specified in the **Result location** field.

A green check mark shall appear next to the conversion job name upon its successful completion:

The screenshot shows the J2CJ configuration window with the following settings:

- J2CJ execution JDK:** 17 Amazon Corretto 17.0.16
- Result location:** J2CJ-output
- Translation mode:** Code style first (selected), Preserve semantics
- Translation scope:** Whole project (selected), Modules
- Module name:** my_demo
- Module version:** 1.0.0
- ☐ Include test sources
- > Additional Options
- Buttons: Build Project, Check, Convert, Create translation plan
- Status: Job #0: Module 'noLombokDemo' translation (with Open Log link)

NOTE: The plugin automatically feeds into J2CJ any files with **mappings** definitions that are located in the source root(s) of the project or module that you are converting. See *Adding Custom Mappings* for details.

The following optional settings appear under **Additional Options**:

- Increase the **Expression depth limit** if J2CJ output contains a few “Expression is too deep” markers. **WARNING:** This may slow down the translation process substantially. Refactoring the Java code is a safer option. See the description of the marker in the *Markers* Chapter for details.

Otherwise, if the translation of the entire project into a single Cangjie module is not desirable, perform the five preliminary steps above, and then select the **Modules** option in the **Translation Scope** group. The whole-project settings below it shall be replaced with a Cangjie output module pane containing the same fields and an editable list of Java modules, pre-filled with all project modules. The button **Add Output Cangjie Module** shall also appear below that pane.

To distribute J2CJ output across multiple Cangjie modules, repeat the following steps as many times as needed:

6. Click the **Add Output Cangjie Module** button. Another Cangjie output module pane shall appear just above it, with an empty list of Java modules.
7. Enter the desired name and version for the output Cangjie module you have just added respectively in its **Module name** and **Module version** fields. (For the very first output module, those will be copied from whole-project settings; you can edit them too.)

J2CJ x

Result location: J2CJ-output

Translation mode: ☒ Code style first ? ☐ Preserve semantics ?

Translation scope: ☐ Whole project ☒ Modules

Cangjie Modules

Module name: J2CJ_demo

Module version: 1.0.0

Java Module	Status
lombokDemo	OK
t1	OK
t2	OK

☒ Include test sources

> Additional Options

Module name: <top-level package name>

Module version: 1.0.0

Java Module	Status
This Cangjie output module contains no Java modules. Add Java modules	

☐ Include test sources ?

▼ Additional Options

Expression depth limit: 5 ?

8. For each Java module that you want to be translated into the newly added output Cangjie module:
 - a. If that Java module currently appears in the list of Java modules in some other output Cangjie module pane, select its name and click the minus “-” button above the list. The plugin shall remove that module from the list.

- b. Click the plus “+” button above the newly added output Cangjie module, select the module you have just removed and click **OK**. The plugin shall conduct a quick validation procedure.
9. Optionally, if the **Include test sources** checkbox is now enabled for the output Cangjie module, check it to also convert the source code files that the IDE has recognized as tests.

Once you are satisfied with the distribution of the input Java modules across the output Cangjie modules, invoke the translator as usual:

10. It is highly recommended to conduct a normal build of the project before attempting translation. If you have not built the project recently (or perhaps never), click **Build Project** to initiate the build process, then wait for it to complete.
11. Click **Check**. The plugin shall conduct a dry-run without generating any output files. If the check has detected any problems, wait 30-60 seconds to let the IDE finish updating its internal indices, and repeat the check attempt. **NOTICE:** If your project uses Gradle, you may also use the “Gradle project sync” feature in the IDE to force/speedup the update.
12. Once the check succeeds, click **Convert** to begin the translation process.

J2CJ messages will be displayed in the **Run** pane. If the **Generate Markdown summary report file** box is checked in **plugin settings**, J2CJ will also generate a *separate* translation report for *each output Cangjie module* (see above for more information about translation reports).

For each output Cangjie module, the following settings appear under **Additional Options**:

- Increase the **Expression depth limit** if J2CJ output contains a few “Expression is too deep” markers. **WARNING:** This may slow down the translation process substantially. Refactoring the Java code is a safer option. See the description of the marker in the *Markers* Chapter for details.
- Should you change your mind later, you may remove the given output Cangjie module by clicking **Remove This Module** button. You cannot remove the only remaining output module.

WARNING: The output module gets removed regardless of whether its list of associated Java modules is empty or not. If that list was not empty, you need to add those Java modules to the remaining output Cangjie modules if you want J2CJ to translate them.

Finally, if you (also) want to skip certain modules of your Java/Android project during translation completely, select each of them in the list of Java modules where it appears and click the minus “-” button above that list.

4.2 J2CJ Command Line

First, you need to figure out how to compile the Java source code that you want to convert to Cangjie using the conventional JDK compiler, `javac`, on the command line. **IMPORTANT:** All that source code must compile in a single `javac` invocation, without using a build automation system such as Maven or Gradle. You may want to use the verbosity options of the system you are using.

Once you’ve obtained that “build-all” `javac` command line, edit it as follows:

- Keep all `javac` arguments in place
- Replace *only the “javac” command itself* with “`java -ea -jar J2CJ-jar`”, where `J2CJ-jar` is the pathname of the `j2cj.jar` file found in your J2CJ **installation**.

Example:

```
javac -cp lib.jar App.java
```

becomes

```
java -ea -jar /opt/j2cj/j2cj.jar -cp lib.jar App.java
```

The edited command line will invoke J2CJ, which will attempt to translate your Java source code to Cangjie.

Here is a more involved example in Bash:

```
# Collect the pathnames of all Java source files located under ./src:
find ./src -type f -name "*.java" > sources.txt

# Do the same for their jar file dependencies collected under ./lib
# and join their pathnames together into a single classpath string:
export JARS="`(readarray -t ARRAY < <(find ./lib -type f -name "*.jar"); IFS=":");
echo "${ARRAY[*]}")`"

# Run J2CJ to convert all source files, placing the result in ./j2cj-output
# using "test" as the module name:
/path/to/JDK/17/bin/java -ea -Dmodule.name=test -jar /path/to/j2cj/j2cj.jar \
-d ./j2cj-output -classpath $JARS @sources.txt
```

NOTE: Generally, for the above script to work you must also provide **mappings** for the dependencies. See [Adding Custom Mappings](#) and [Incremental Translation](#) for details.

4.2.1 Command-Line Options

The translator *recognizes* all command-line options supported by the standard `javac` Java to bytecode compiler as of OpenJDK 17, but *ignores* any options that are not applicable, such as options affecting the contents of the generated class files, which J2CJ obviously does not generate. Options for controlling annotation processors are another notable example of recognized-but-ignored options, as J2CJ support for annotations is **very limited**.

For example, if you are translating Java source code written for a version of the JDK that is older than the version on which J2CJ runs, you can use the options `--system` and `--release`, just as if you were compiling for that older JDK with a newer `javac` compiler:

```
/path/to/JDK/17/bin/java -ea -Dmodule.name=legacy -jar /path/to/j2cj/j2cj.jar \
--system /path/to/JDK/11 --release 11 \
-d ./j2cj-output -classpath $JARS @sources.txt
```

Similarly, if the original Java project is modularized, you can use the `--module-source-path` and `-m` options instead of supplying a list of individual `.java` files.

In addition to all those `javac`-compatible options, J2CJ also provides options that control translation.

IMPORTANT: As of the current version, most J2CJ-specific command-line options have to be passed as JVM properties, that is, *before* the `-jar` option of the `java` launcher using the launcher's `-D-` option:

```
java -ea -Dmodule.name=quux -jar /opt/j2cj/j2cj.jar -cp lib.jar App.java
```

The following command-line options are specific to J2CJ:

```
-Dgenerate.mapping.file=pathname
```

Save mappings for the code that is being translated into a file with the given *pathname*. See *Incremental Translation* for details.

`-Dmodule.name=identifier`

Use *identifier* as the module name in the generated Cangjie package declarations.

Modules were only added to Java in version 9, so there is plenty of legacy Java code that does not use modules. In contrast, each Cangjie source file belongs to a certain named package, which in turn belongs to a certain module. By default, J2CJ uses the leftmost identifier of those constituting the Java package name as the module name, so the Java package declaration

```
package cu.mber.some;
```

becomes

```
package cu.cu.mber.some
```

in the translated Cangjie file. This option enables you to specify a particular module name to use in place of the default.

`-Dmodule.version=string`

Use *string* as the module version number in the generated `cjpm.toml` file. The default is “1.0.0”.

WARNING: J2CJ does *not* check whether *string* is a correct version number from the CJPM standpoint.

`-Dextra.log=pathname`

`-Dextra.log=true`

Write a translation summary report in Markdown format to a file with the given *pathname*, or, if the option is set to **true**, to a file named `log-stats.md` located in the current working directory.

`-Dmax.expression.depth=N`

Increase the maximum expression nesting/chaining level to suppress the generation of the “Expression is too deep” diagnostic marker for overly complex expressions. The default value of *N* is 5.

WARNING: High values of *N* may *significantly* increase the time of translation. If the “Expression is too deep” marker appears often in J2CJ output, consider breaking the original Java expressions down into two or more parts before the next translation attempt instead of raising this limit.

The complexity of translating chained expressions of the form `a.f().g().h().k()...` and deeply nested expressions grows exponentially with each level of chaining/nesting, which may push the total translation time beyond acceptable. The translator therefore limits the depth of expressions that it can handle.

`-mode=semantic|codestyle`

Translation mode. The default value is `codestyle`, which makes J2CJ produce shorter and more readable Cangjie source code at the cost of losing or altering certain semantics of the original Java program. Each code portion that needs post-translation analysis and editing would contain a **marker** or simply won't compile.

By way of example, in Java the equality test operators `==` and `!=` accept arguments of any compatible reference types, because the semantics of those operators is a simple reference comparison. In Cangjie, however, you can only test a value for equality if the operator function `==` is defined for its type. Moreover, that implementation may compare references, values of some or all member variables, hash sums, or something else.

So while J2CJ translates those Java operators into their Cangjie counterparts in the default mode (`-mode=codestyle`), the resulting code won't compile.

Setting `-mode` to `semantic` inflicts the following changes in the generated code:

- Synthetic `Equatable<T>`, `Hashable`, and `ToString` implementations are added to all Cangjie classes generated by J2CJ.
- The `==` and `!=` operators are translated into calls of the `std.core.refEq` function if the types of their arguments are Java reference types.
- The `open` modifier is added to all non-abstract classes, interfaces, and member functions if the original Java entities are not `final`, regardless of whether they have any known to J2CJ subclasses, subinterfaces, or overriding methods respectively.
- A helper library is used to enable or improve the accuracy of translation of certain Java language constructs and core library functions that are absent in Cangjie or have different semantics.

Refer to [Semantics Preservation](#) for a more detailed discussion of the above translation rule alterations.

`--enum-functions=list`

Force the generation of certain Java enum methods in the default “code style first” mode (`-mode=codestyle`).

In that mode, J2CJ does *not* normally synthesize the equivalents of certain built-in methods of Java enumeration types, namely `values`, `valueOf`, `name` and `ordinal`, when translating a Java enum type into a Cangjie enum. `list` is a comma-separated list of one or more names of those Java methods.

Example:

`--enum-functions=values,ordinal`

In the `semantics preservation` mode (`-mode=semantic`), this option has no effect. All four functions are always present in the resulting class.

See also [Enum Types](#).

4.3 Advanced Usage

J2CJ may be unable to translate certain parts of your Java program due to the complexities discussed in the [Preface](#). It would still try to produce “approximately correct” Cangjie source code for such parts where possible, but wrap it in the so called *markers* `<-- -->` with an error message, so it won't compile.

J2CJ bugs aside, there are two main reasons for the translation to fail:

1. Your source code uses a Java *language feature* that J2CJ does not support (yet). See [Java Features Not Supported by J2CJ](#) for the current list of such features and implementation prognosis.

Example:

There is no proper equivalent for Java instance initializers in Cangjie, so J2CJ would translate the following Java class

```
class Foo{
    int x;
    {
        x = 1;
    }
}
```

into

```
open class Foo <: Hashable & ToString {
    init() { }

    var x: Int32 = 0

    <-- Unsupported initialization blocks: {
        x = 1i32
    } -->

    . . .
}
```

Resolution: Change your Java source code so that it no longer uses the unsupported feature. Test your changes to confirm that you did not accidentally break your Java code, then re-run it through J2CJ.

In the above example, it would be easy to replace the assignment in the instance initializer with a field initializer:

```
class Foo {
    int x = 1;
}
```

and J2CJ would then happily yield

```
open class Foo <: Hashable & ToString {
    init() { }

    var x: Int32 = 1i32

    . . .
}
```

WARNING: Poorly written legacy code may trip you up where you least expect that. Applying the above transformation to

```
class Foo {
    int x;           // x gets initialized with 0 by default
    . . .           // Many lines of code
    int y = x + 1;    // y == 1
    . . .           // More lines of code
    {
        x = -1;       // Now x == -1
    }
    . . .           // Even more lines of code
}
```

without noticing the use of `x` in the initializer of `y` would introduce a bug:

```
class Foo {
    int x = -1;       // x == -1
    . . .
    int y = x + 1;    // Now y == 0 instead of 1. Oops!
```

```
}
. . .
```

2. Your source code uses a Java Platform *API* or *third-party library* that J2CJ knows nothing about.

Resolution: Either

- **Add mappings** for the offending API or library, or
- Change your Java source code to use an API/library already known to J2CJ instead, test your changes, and then feed your Java sources into J2CJ again.

Of course, in both cases you can also manually write the Cangjie code for the portions of your Java source that J2CJ could not handle and exclude those portions from its input. It is up to you to determine what would be the most effective and/or least error-prone course of action.

IMPORTANT: If you make *any* changes to your Java source code, whether to enable the automated translation of parts that use unsupported Java language features or APIs/libraries for which no mappings exist, make sure to fully test it before feeding into J2CJ again.

To illustrate the importance of such testing, suppose in the above example there was more complex code computing the initial value of `x` in the instance initializer, and the initializer expression of another field referred to `x`.

```
class Foo {
    int x;
    {
        // Some code that initializes x
    }
    int y = x + 1;
}
```

In Java, instance field initializers and instance initializer blocks are executed sequentially in lexical order. If you move the `x` initialization code to the parameterless constructor, you must also move there the initialization of `y`, which otherwise may be initialized with an invalid value:

```
class Foo {
    int x;
    Foo () {
        // Some code that initializes x
    }
    int y = x + 1;    // y == 1 no matter what !!!
}
```

4.3.1 Incremental Translation

It is often desirable to translate a large project module-by-module or otherwise break down the process into several stages. Another scenario when incremental translation becomes useful, is multiple projects sharing certain libraries, modules or packages.

As Cangjie does not permit circular dependencies, neither does J2CJ. Any such dependencies in the original Java code must be refactored away before translation. After that, it is always possible to follow the incremental translation process:

1. Break down the source code that is subject to translation into parts that form a dependency graph with no cycles.

2. Identify the parts that do not depend on other parts.
3. Translate those “independent” parts separately and test the translated code.
4. Identify the parts that only depend on the previously translated parts.
5. Translate those newly identified parts and test the translated code.
6. If there are still any non-translated parts left, go to step 4.

However, for step 5 to succeed *without re-translating the already translated parts*, J2CJ must take additional input: the *actual mappings* for those parts. Without those mappings, J2CJ will be unable to properly translate any Java code depending on the parts that have already been translated.

To facilitate the incremental translation process described above, J2CJ can optionally save the mappings for the code it is translating into a `.cjmap` file, which can then be supplied as input when translating the dependent parts of the project.

To have J2CJ generate a `.cjmap` file for the translated set of Java source files, pass the desired pathname of that file over via the `generate.mapping.file` system property, e.g. `-Dgenerate.mapping.file="./common.cjmap"`

NOTE: System properties are passed as arguments of the `java` command, not as arguments of J2CJ itself.

Then, when translating a dependent set of Java source files, include that generated `.cjmap` file into `root.cjmap`.

Example:

Translate some common modules:

```
java -ea -Dgenerate.mapping.file="./common.cjmap" -jar j2cj.jar \
  -d "./common" @common.lst
```

Now add their mappings when translating other modules:

```
#include "stdlib.cjmap"
#include "mappings.cjmap"

#include "custom.cjmap"
#include "common.cjmap"
```

4.3.2 Semantics Preservation

By default, J2CJ strives to generate idiomatic Cangjie source code even at the cost of losing or altering certain semantics of the original Java code. Cangjie sources produced in that “code style first” mode are generally smaller and more readable/maintainable, but may also contain more **markers** and require more post-translation analysis and editing.

In the semantics preservation mode (`-mode=semantic`), J2CJ translation rules are changed as follows:

1. All Java classes either implement or inherit methods `equals`, `hashCode` and `toString`, as they are present in the root of the Java class hierarchy, `java.lang.Object`. But the Cangjie equivalents of those methods are found in the `Equatable<T>`, `Hashable`, and `ToString` interfaces respectively. If the `-mode=semantic` option is specified, J2CJ adds those three interfaces to all converted classes and synthesizes their implementations. This makes the converted Cangjie code noticeably bigger, but at the same time less likely to break *if* the original Java code relied on the presence of any or all of those methods.

2. The Java equality test operators `==` and `!=` applied to values of reference types compare the references themselves. J2CJ translates them verbatim by default. However, those Cangjie operators may be undefined for types that do not implement the `Equatable<T>` interface. Moreover, for each particular reference type they may be implemented as a comparison of references, values, partial values, hash sums, or something else. When the `-mode` option is set to `semantic`, J2CJ translates `a == b` to `refEq(a, b)` if `a` and `b` have Java reference types in the original code.
3. J2CJ normally omits the `open` modifier on classes and interfaces that have respectively no known subclasses and subinterfaces, as well as on non-overridden/non-redefined member functions, even if the original Java types and methods are not `final`. With `-mode=semantic`, it marks each Cangjie interface, non-abstract class and member function as `open` unless the original Java type or method is `final`.
4. A helper library, `j2cjlib`, is used to improve the semantic accuracy and completeness of translation. In particular, certain complex mappings either cannot be implemented at all without using such functions, or their implementation is less accurate and/or efficient in the default “code style first” mode.
5. Function types are not used for translating functional interface usages, and lambdas are always translated into classes.
6. **Changed in 1.7.2:** Types of `volatile` fields other than `boolean` and integer primitive types are translated differently. See [Types of volatile Fields](#) below for details.
7. **New in 1.7.2:** J2CJ conducts more strict checks when determining whether a basic `for` statement may be translated into a Cangjie `for-in` loop. See [Basic for Statements](#) for details.

4.3.2.1 Types of volatile Fields

Changed in 1.7.2:

The types of `volatile` fields are translated into the respective atomic types from `std.sync`. Reading from and writing to such fields is translated into the calls of `load` and `store` member functions of those types.

The mapping for reference types depends on the translation mode due to the `Option<T>` wrapping in the semantics preservation mode (`-mode=semantic`). **NOTE:** As atomic classes for character and floating-point numeric types are not provided in `std.sync` (yet), they have to be wrapped into containers, effectively becoming reference types.

In the below tables, `X'` is the Cangjie type to which J2CJ translates the Java type `X` used in contexts other than `volatile` field type.

In the default “code style first” mode (`-mode=codestyle`), types of `volatile` Java fields are translated as follows:

Java	Cangjie
<code>boolean</code> / <code>Boolean</code>	<code>AtomicBool</code>
<code>byte</code> / <code>Byte</code>	<code>AtomicUInt8</code>
<code>short</code> / <code>Short</code>	<code>AtomicInt16</code>
<code>int</code> / <code>Integer</code>	<code>AtomicInt32</code>
<code>long</code> / <code>Long</code>	<code>AtomicInt64</code>

Java	Cangjie
char / Character	AtomicReference<Box<Rune>>
float / Float	AtomicReference<Box<Float32>>
double / Double	AtomicReference<Box<Float64>>
String	AtomicReference<Box<String>>
class C	AtomicReference<C'>
interface I	AtomicReference<I'>
T[]	AtomicReference<Box<Array<T'>>>

In the **semantics preservation** mode (`-mode=semantic`), types of **volatile** Java fields are translated as follows:

Java	Cangjie
boolean	AtomicBool
byte	AtomicUInt8
short	AtomicInt16
int	AtomicInt32
long	AtomicInt64
char	AtomicReference<ValueBox<Rune>>
float	AtomicReference<ValueBox<Float32>>
double	AtomicReference<ValueBox<Float64>>
Boolean	AtomicOptionReference<ValueBox<Bool>>
Byte	AtomicOptionReference<ValueBox<UInt8>>
Short	AtomicOptionReference<ValueBox<Int16>>
Integer	AtomicOptionReference<ValueBox<Int32>>
Long	AtomicOptionReference<ValueBox<Int64>>
Character	AtomicOptionReference<ValueBox<Rune>>
Float	AtomicOptionReference<ValueBox<Float32>>
Double	AtomicOptionReference<ValueBox<Float64>>
String	AtomicOptionReference<ValueBox<String>>
class C	AtomicOptionReference<C'>
interface I	AtomicOptionReference<I'>
T[]	AtomicOptionReference<ValueBox<Array<T'>>>

NOTE: If T is a Java reference type other than array, T' is the respective Cangjie type wrapped in **Option**.

4.3.2.2 Function Types

In the default “code style first” mode (`-mode=codestyle`), uses of a Java functional interface type annotated with `@FunctionalInterface` are translated into uses of the equivalent Cangjie function type, unless either or both of the following is true:

- The sole abstract method of the interface is parameterized.

NOTE: A method is considered parameterized if its declaration contains one or more type *parameters*, even if its signature contains no type *variables*:

```
@FunctionalInterface
interface FI {
    <T> int foo(int arg); // Parameterized despite absence of type variables
}

@FunctionalInterface
interface PFI <T> {
    int bar(int arg); // Not parameterized (even though the interface is)
}
```

- A function type cannot be defined because the interface type declaration is recursive:

```
@FunctionalInterface
interface Recursive {
    Recursive f();
}
```

For example, uses of `java.lang.Runnable` are translated into `() -> Unit`, uses of `java.util.function.Function<String, String>` into `(String) -> String`, and so on.

Consequently, any invocations of the abstract method of a functional interface translated in this manner are translated into calls of functions.

Declarations of functional interfaces are still translated regardless of the mode, but the annotation `@FunctionalInterface` is not preserved.

Definitions of classes that implement functional interfaces (including anonymous classes) are translated normally, but any uses of *instances* of those classes in contexts where a value of the respective function type is now expected are translated into member function references.

```
@FunctionalInterface
interface DoubleFunc {
    double f(double x);
}

class Halver implements DoubleFunc {
    public double f(double x) { return x / 2.0; }
}

...
static double apply(DoubleFunc df, double x) {
    return df.f(x);
}
...
double res1 = apply(x -> x * 2.0, 10.0);
double res2 = apply(new Halver(), 10.0);
```

```
interface DoubleFunc { // Interface declaration is preserved
    func f(x: Double): Double
```

```

}

class Halver <: DoubleFunc {
  override public func f(x: Double): Double {
    return x / 2.0;
  }
}

. . .
// Parameter 'df' has function, not interface type...
static func apply(df: (Double) -> Double, x: Double): Double {
  df(x) // ...and is called directly
}

. . .
let res1 = apply({x => x * 2.0}, 10.0)
let res2 = apply(Halver().f, 10.0)

```

4.4 Translation Result

The last line of J2CJ output provides translation statistics: the number of lines of code successfully generated by the translator, the total number of lines in its output, which is essentially the first number plus the number of lines with **markers**, and the ratio of the first two numbers:

```
Translation ratio: 128/130 lines (98,5%)
```

If the translation was unsuccessful (i.e. the ratio is less than 100%), you need to review the generated code and act on each marker. See *Advanced Usage* for details.

If the translation was successful, which is indicated by 100% translation ratio, you can compile the generated Cangjie sources using the standard `cjpm` tool as follows:

1. The translator places a `cjpm.toml` file in the root directory of the generated module:

```

[dependencies]
[dependencies.j2cjlib]
  path = "../j2cjlib"
  version = "1.0.0"

[package]
  . . .

```

NOTE: The dependency on the `../j2cjlib` module is only present if J2CJ was run in the **semantics preservation** mode (`--mode=semantic`). That module is essentially a helper library included with the translator. It provides types and functions for which there are no equivalents in the standard Cangjie library. You need to either copy it from your J2CJ **installation** into the J2CJ output directory or create a symbolic link:

```
ln -s /path/to/j2cj/j2cjlib/ ../j2cj/output/dir
```

2. Now you may simply run `cjpm build` in the directory that contains the `cjpm.toml` file for the translated sources.

5 Mappings

The translation rules for supported Java language constructs: declarations, statements, operators, primitive types, etc., are quite complex and therefore beyond user control. At the same time, direct equivalents of many core Java API classes and interfaces such as `String`, `HashMap` or `Iterable` exist in the Cangjie standard library, so the former can be simply *mapped* onto the latter.

A *mapping* associates a Java type, or member thereof, with its closest Cangjie equivalent, so that any uses of that type or member can be properly translated. J2CJ includes mappings for a number of core Java Platform API classes, and you can also add mappings for your own classes and/or third-party libraries to increase the percentage of automatically translated code.

As of the current version, user-defined mappings can be thought of as simple name mappings: they require Cangjie methods to have the same number of (mandatory) parameters listed in the same order as respectively the number and order of the formal parameters of the original Java method.

NOTE: The exact semantics of the methods may differ. For instance, the parameterless method that returns the length of the given instance of `java.lang.String` is called `length()`, whereas the respective method of the Cangjie `String` type is called `size()`. However, the Java version returns the number of UTF-16 code units, whereas the Cangjie version returns the length of the UTF-8 representation of the string. While the mapping of `length` onto `size` preserves invariants such as `(s1 + s2).length() == s1.length() + s2.length()`, the Java code relying on the *exact* semantics of the `String.length` method may become incorrect after translation.

There are also built-in, more sophisticated mappings, for certain frequently used core Java API types and methods that do not have such straight equivalents in Cangjie.

5.1 Mappings Loading Process

Upon launch, J2CJ looks up a file called `root.cjmap` in its current working directory. If there is no such file, it loads and parses the built-in version from its own jar file, `j2cj.jar`.

A `root.cjmap` file consists of two separate mandatory parts:

- Definitions of Cangjie types and members (examples are in the `stdlib.cjmap` file, embedded inside `j2cj.jar`)
- Definitions of Java mappings (examples are in the `mappings.cjmap` file, embedded inside `j2cj.jar`)

The default `root.cjmap` file simply includes the respective files:

```
#include "stdlib.cjmap"
#include "mappings.cjmap"
```

NOTE: The **IntelliJ IDEA Plugin** additionally looks up `.cjmap` files in all source roots of the project or module that is being converted and passes their pathnames over to J2CJ.

5.2 Adding Custom Mappings

Developers may add their own mappings using either of the following two methods, or even both of them:

Method 1:

Pass the pathnames of `.cjmap` files to J2CJ as normal command-line arguments (unlike the **J2CJ-specific command-Line options** that have to be passed as system properties):

```
/path/to/JDK/bin/java -Dmodule.name=App -jar /path/to/j2cj/j2cj.jar \
  App.java custom.cjmap
```

J2CJ loads those `.cjmap` files in the order in which they appear on its command line, after it completes loading the `root.cjmap` file.

The relative order of `.java` and `.cjmap` files does not matter; they can be mixed freely.

NOTE: The **IntelliJ IDEA Plugin** automatically passes to J2CJ the pathnames of all `.cjmap` files located in the source root(s) of the project or module that is being converted.

Method 2:

- Create a *local* `root.cjmap` file in the directory from which you launch J2CJ, with the same content as the default one:

```
#include "stdlib.cjmap"
#include "mappings.cjmap"
```

- Create another `.cjmap` file with your own mappings in the same directory. Let's say you named it `custom.cjmap`.
- Append the line

```
#include "custom.cjmap"
```

to the local `root.cjmap` file.

When you migrate to new J2CJ versions later, simply check if there were any changes to the default `root.cjmap`, and in the unlikely event that there indeed were changes, reflect them in your local `root.cjmap` file.

This method ensures that the default mappings will continue to load.

5.3 Mapping Syntax

Cangjie qualified names have the following format: `moduleName.packageNameSeparatedByDots.packageMember`. Usually `packageMember` is a type name.

5.3.1 Cangjie Declarations

The syntax of Cangjie definitions in J2CJ mappings files is close to a subset of Cangjie syntax, but requires full qualification of each type name as defined **above**. Of the elements that are not actually needed for translation, it excludes the bulkier ones, which means there are:

- No imports
- No method bodies
- No field initializers
- No `var` or `let` keywords

Access modifiers and annotations can be present but are ignored, so copying/pasting definitions from Cangjie source files is a little less tedious than it could be.

5.3.1.1 Cangjie Declaration Examples

The below examples are excerpts from the `stdlib.cjmap` file that is part of the the `j2cj.jar` archive. Developers may want to study that file for more examples and inspiration.

The `Option` enumeration:

```
enum std.core.Option<T> {
  getOrThrow(): T
  isSome(): Bool
  isNone(): Bool
  static func Some(value: T): Option<T>
  static func None(): Option<T>
}
```

`Iterable` and `Iterator` interfaces:

```
interface std.core.Iterable<E> {
  iterator(): std.core.Iterator<E>
}

interface std.core.Iterator<E> <: std.core.Iterable<E> {
  next(): Option<E>
}
```

The `Collection` interface:

```
interface std.core.Collection<T> <: std.core.Iterable<T> {
  size: Int64
  isEmpty(): Bool
  toArray(): Array<T>
}
```

The `Array` structure with `ToString` extension:

```
struct std.core.Array<T> <: std.core.Collection<T> {
  copyTo(Array<T>, Int64, Int64, Int64): Unit
  clone(): Array<T>
}
extend std.core.Array<T> <: std.core.ToString where T <: std.core.ToString { }
```

`ToString` itself:

```
interface std.core.ToString {
  toString(): String
}
```

A class:

```
class std.sync.Monitor {
  wait(): Bool
  notify()
  notifyAll()
}
```

Package-level function declarations:

```
package std.math {
  func cbrt(x: Float64): Float64
  func cbrt(x: Float32): Float32
  func cbrt(x: Float16): Float16
}
```



```
func logBase(x: Float32, base: Float32): Float32
    . . .
}
```

5.3.1.2 Extension Declarations

Extensions can only be declared in the bodies of **package** declarations corresponding to the real owning packages in Cangjie. (Here the mapping file syntax deviates from the Cangjie syntax with its bodyless package declarations.) J2CJ will automatically insert imports of all necessary interface types when an **extend** member is used, as required by the Cangjie language. Any type variables used in the declaration must be declared as part of the **extend** declaration (after the **extend** keyword). Refer to the below excerpt from the `stdlib.cjmap` file included with J2CJ for illustration.

```
. . .
public class std.io.StringReader<T> where T <: std.io.InputStream {
    public init(input: T)
    . . .
}
. . .
package std.io {
    extend<T> std.io.StringReader<T> <: Resource where T <: Resource {
        public func close(): Unit
        public func isClosed(): Bool
    }
    extend<T> std.io.StringReader<T> <: std.io.Seekable where T <: std.io.Seekable {
        public func seek(sp: std.io.SeekPosition): Int64
    }
}
. . .
package j2cjlib.utils {
    . . .
    extend<T> std.io.StringReader<T> <: j2cjlib.utils.J2CjStringReaderExtension {
        public func read(buf: Array<Rune>): Int32
    }
    . . .
}
```

5.3.2 Java Mappings

The second part of a mappings file contains the definitions of Java to Cangjie mappings. Any use of a Java class or interface, the definition of which is *not* present in the source code files supplied as input to J2CJ, would fail to translate unless there is a mapping for:

- The type itself (we only support the same number, order and names of type parameters), except when only its **static** members need to be mapped to package-level functions and/or global variables.
- The member or constructor that is actually used in the source code constituting J2CJ input.

NOTE: Certain core Java API methods cannot be mapped/translated at all. Consider the method `java.util.Iterator<E>.hasNext()`. The Cangjie version of that interface, `core.Iterator<E>` relies on `Option` to communicate the absence of an element to the caller. And a naive implementation `next().isSome()` would consume the element.

5.3.2.1 Type Mappings

The syntax of a single Java type mapping is as follows:

```
'mapping' Java-type [ type-arguments ] [ '=>' Cangjie-type [ type-arguments ] ] '{'
  { Java-member [ '=>' Cangjie-member ] }
'{'
```

where `Java-type` and `Cangjie-type` are fully qualified names of a Java type and a Cangjie type respectively. Qualification may be omitted for Cangjie types defined in the `std.core` package.

The `'=>'Cangjie-type [ type-arguments ]` part is made optional in order to support the mapping of Java classes that are essentially *namespaces*: they only have **static** members and no user-accessible constructors, and are therefore never instantiated in the translated code. The equivalent functionality on the Cangjie side is often provided by global variables and top-level functions, so there is simply no Cangjie class to map to. A well-known example of such Java class is `java.lang.Math`:

```
mapping java.lang.Math {
    . . .
}
```

NOTE: It is permitted to map a Java interface to a Cangjie class, but *you should only ever do that as a workaround* for the absence of the respective interface in Cangjie code.

Technically, the reverse is also true: you can map a Java class to a Cangjie interface, but such mapping is of little use as the instantiations of that class won't translate.

5.3.2.2 Member And Constructor Mappings

In the simplest case, the target `Cangjie-type` has a member with the same name and semantics, so the respective member mapping consists of just a name:

```
mapping java.lang.Iterable<T> => std.core.Iterable<T> {
    iterator
}
```

Similarly, the mapping for a Java constructor that has a match in the target Cangjie type consists of just the string `<init>`:

```
mapping java.util.HashSet<E> => std.collection.HashSet<E> {
    <init>
}
```

If the respective `Cangjie-type` member has a different name, that name can be specified after `=>`:

```
mapping java.util.Set<E> => std.collection.Set<E> {
    add => put
    addAll => putAll
    . . .
}
```

In certain cases, `Cangjie-member` can be a fully qualified name, which enables mapping to a member of some other type, or even to a global variable or a top-level function of a Cangjie package. This is mostly useful for mapping **static** Java fields and methods:

```
mapping java.lang.Math {
    abs => std.math.abs
    sqrt => std.math.sqrt
    exp => std.math.exp
}
```

. . .

IMPORTANT: The target method or constructor may have different, but compatible, parameter types, and/or, in case of a method, return type. The translator will insert the appropriate type casts automatically.

5.4 Additional Method And Constructor Mapping Considerations

5.4.1 Mapping Overloaded Methods And Constructors

The **above compact mapping forms** work for overloaded Java methods and constructors as well. **However, they should be used with caution, as its use may introduce latent issues.**

It is often the case that not all overloaded Java methods/constructors have equivalents in the target Cangjie class. Consider, for instance, the `HashSet` class mapping:

```
mapping java.util.HashSet<E> => std.collection.HashSet<E> {
    <init>
}
```

Any uses of the constructor `HashSet(int initialCapacity, float loadFactor)` in the Java code fed to J2CJ would translate into a **marker**, as there is no matching constructor in the Cangjie `HashSet` class. But if that constructor is *never used* in the said Java code, the non-existence of its Cangjie equivalent won't trigger a translation error, and the current version of J2CJ won't issue a warning either.

It is therefore safer to specify mappings for overloaded methods and constructors one-by-one, using **signatures** to distinguish them. That is also the only option if the Cangjie equivalents of overloaded Java methods have different names, i.e. are not overloaded.

Example:

The Cangjie threads API differs radically from the Java one, so the J2CJ helper library comes to the rescue. It does not support thread groups, though, so as of Java 21 only four constructors of `java.lang.Thread` out of nine can be mapped:

```
mapping java.lang.Thread => j2cjlib.mappings.sync.J2CjThread {
    <init>()
    <init>(java.lang.Runnable)
    <init>(java.lang.String)
    <init>(java.lang.Runnable,java.lang.String)
    . . .
}
```

5.4.1.1 Java Signatures

As you have already seen from the above examples, the signatures of Java methods and constructors in the mapping files don't look like JVM signatures, but more like Java method headers, with dots instead of slashes as package name separators, full names of primitive types, arrays denoted as `[]` suffixes, and commas as parameter separators:

- `(java.lang.Object,java.lang.Integer)int` instead of `(Ljava/lang/Object;Ljava/lang/Integer;)I`,
- `(byte[])long` instead of `([B)J`,
- etc.

Note also that only method signatures must end with either a return type or **void**, as constructors don't return values by definition.

All signatures are erased according to the Java Language Specification: each type variable is replaced with its upper bound, and all type parameter lists are removed. For example:

- The `java.util.Collection<? extends E>` parameter type of a `java.util.HashSet` constructor becomes simply `java.util.Collection`,
- The `T[]` parameter type of the `java.util.Arrays.parallelSort` method with `T extends java.lang.Comparable<? super T>` generic constraint becomes `java.lang.Comparable[]`,
- and so on.

5.4.2 Mapping Inherited Methods

IMPORTANT: There is no need to define mappings for inherited methods. In fact, doing so *without* mapping the “origin” of the method would cause the translator to fail. This is a technical limitation that may be removed in future versions.

To reiterate, the mapping(s) for the given method *must* be provided only for classes and interfaces that either *introduce* or *override* that method. You may include mappings for the types merely inheriting that method as well, but as of the current version they will be ignored.

This in particular means that you may need to define mappings for non-**abstract** methods of **abstract** classes, even if your program only uses the implementing classes.

Example:

```
abstract class AbstractFoo {
    abstract int foo();
    boolean bar() { return false; }
    char baz() { return 'A'; }
}

class ConcreteFoo extends AbstractFoo {
    int foo() { return 1; }
    @Override
    boolean bar() { return true; }
}
```

Even if the program being translated only ever uses the `ConcreteFoo` class, the following mapping alone is incomplete and superfluous at the same time:

```
mapping ConcreteFoo => CJ_ConcreteFoo_Equivalent {
    foo
    bar
    baz
}
```

The reason is that by the time the translator would look up the applicable mapping for `.baz()` in a code fragment like `new ConcreteFoo().baz()`, it will already have determined that the method being invoked is actually `AbstractFoo.baz()`, and hence look for a mapping of `baz` within the mappings of `AbstractFoo` members.

The correct mapping for the above example looks like this:

```
mapping AbstractFoo => CJ_AbstractFoo_Equivalent {
    baz
}

mapping ConcreteFoo => CJ_ConcreteFoo_Equivalent {
    foo
    bar
}
```

5.5 More Simple Mapping Examples

For more examples and inspiration, look into the default `mappings.cjmap` file included in `j2cj.jar`

NOTE: That file is far from being complete. The current version of J2CJ is mostly focused on the Java Platform API, and often uses the much more complex internal mapping system to enable the translation of the ubiquitous parts of that API that do not have straight, 1:1 equivalents in the Cangjie standard library (yet). Therefore that file does not represent each and every mapping that J2CJ provides out-of-the-box.

6 Annotation Support

J2CJ supports the translation of user-defined annotation types and also of certain standard Java annotations.

6.1 Predefined Annotation Types

- The annotations `@SafeVarargs`, `@Documented`, `@Inherited`, and `@Repeatable` have no equivalents in Cangjie as of today and are therefore translated into **markers**. Should such equivalents appear in the future, you can add simple mappings for them as described in *Annotation Mappings* below.
- The annotation `@Override` is “translated” into the `override` modifier. (quoted because the `override` modifier is mandatory in Cangjie, so it would appear anyway).
- The annotation `@SuppressWarnings` is recognized and skipped.
- The annotation `@Retention` is skipped. All user-defined annotations are assumed to have runtime retention.
- The annotation `@Target` is translated into the `target` parameter of the Cangjie `@Annotation` annotation. Supported element types are: `TYPE`, `ANNOTATION_TYPE`, `FIELD`, `METHOD`, `PARAMETER`, and `CONSTRUCTOR`. Element types `PACKAGE`, `LOCAL_VARIABLE`, `TYPE_PARAMETER`, `TYPE_USE`, `MODULE`, and `RECORD_COMPONENT` are not supported due to the lack of Cangjie equivalents and are translated into markers.
- The annotation `@FunctionalInterface` is taken into account when functional interfaces are translated in the default “code style first” mode (`-mode=codestyle`), but is not propagated into the output Cangjie code in any way. In the **semantics preservation** mode (`-mode=semantic`), it is recognized and skipped. See *Functional Interfaces* for details.
- The annotation `@Deprecated` is translated into its Cangjie equivalent with the same name.

6.2 User-Defined Annotation Types

6.2.1 Annotation Type Definitions

A Java annotation type declaration is translated into a Cangjie annotation class. Each element, which in Java is defined by a `@interface` member method declaration, turns into *two* Cangjie entities with the same name that matches the element name: a member variable and a const init parameter (possibly with default value).

Example:

```
@Retention(value= RetentionPolicy.RUNTIME)
@Target({ ElementType.TYPE, ElementType.FIELD, ElementType.METHOD, ElementType.
    CONSTRUCTOR, ElementType.PARAMETER })
public @interface Anno {
    boolean bool();
    String name() default "name";
    String color() default "color";
}
```

becomes

```
@Annotation[target: [Type, MemberVariable, MemberFunction, Init, Parameter]]
public class Anno {
    let bool: Bool

    let name: ?String

    let color: ?String

    const init(bool: Bool, name!: ?String = "name", color!: ?String = "color") {
        this.bool = bool
        this.name = name
        this.color = color
    }
}
```

6.2.2 Annotation Usages

A Java annotation is translated into an invocation of the `const init` function of the respective Cangjie annotation class:

```
@Anno(bool = true)
public class Annotated {
    @Anno(bool = true, name = "f")
    int field;
}
```

becomes

```
@Anno[true]
public open class TestAnno <: Hashable & ToString {
    @Anno[true, name: "f"]
    var field: Int32 = 0
}
```

6.3 Annotation Mappings

The current J2CJ distribution includes no simple mappings for predefined Java annotation types other than those **explicitly supported**, but you can create them if needed.

To declare a Cangjie annotation class in a `.cjmap` file, use regular Cangjie syntax. In the body, mention all member variables and the signature of the `const init` function:

```
@Annotation class std.core.NonExistentCangjieLabel {
    init(cjvalue!: String = "cjvalue")
    cjvalue: String
}
```

To define a simple mapping of Java library annotation (“simple” = there is an appropriate mapping for every Java annotation element (!!! refine)), use a **regular mapping declaration**:

```
mapping jdk.jfr.Label => std.core.NonExistentCangjieLabel {
    value => cjvalue
}
```

7 Java Features Not Supported by J2CJ

Technically, for any correct Java program there is a correct Cangjie program that would produce the same output if given the same input. However, there is a number of Java language constructs that have no direct equivalents in Cangjie, so any conversion tool must express them using other Cangjie constructs. And even the constructs present in both languages often have peculiarities that would make a straightforward conversion produce invalid results. Finally, there are also features that are difficult to support using the current technology, but support for them is planned in the future.

This section lists such features along with the current status and future plans. Its structure follows that of the Java Language Specification.

7.1 Lexical Structure

7.1.1 Comments

J2CJ attempts to preserve all source code comments, with the following caveats:

- The alignment of comments is not taken into account:

```
private int x = 153      // One hundred fifty-three
    * 2                // times two
    /// 7              // not divided by seven
    -/* 6 */5          // minus six^H^H^Hfive
    /* + randomInt(10) // mix in a random int */
    + 1;               // plus one
```

becomes

```
private var x = 153i32 // One hundred fifty-three
    * 2i32 // times two
    /// 7      // not divided by seven
    - /* 6 */ 5i32 // minus six^H^H^Hfive
    /* + randomInt(10) // mix in a random int */ + 1i32 // plus one
```

- Comment misplacement may occur in certain ambiguous cases. For instance, Java **if** and loop statements do not necessarily have blocks { } as substatements:

```
while(f(x) > epsilon) x /= 2.0;
if (p(y)) do_this(y) else do_that(y);
```

whereas their Cangjie counterparts, conditional and loop expressions, *always* contain blocks.

Now, consider the following two Java statements:

```
if (foo == null) foo = FooFactory.make();      // Make a foo if needed

if (bar != null) myBar = bar
else throw new NullPointerException();        // Should never happen
```

Each of the two line comments is placed after the closing semicolon ; of an **if** statement. However, the first comment relates to the entire statement, whereas the second one actually relates to the **else**-part. The translator has no way to tell those two cases apart, so it always assumes that a line comment *immediately* following a statement relates to that statement in its entirety:


```

if (bar.isSome()) {
    myBar = bar
} else {
    throw NoneValueException()
} // Should never happen

```

Changing the original Java code would remedy that problem:

```

if (bar != null) myBar = bar
else {
    throw new NullPointerException(); // Should never happen
}

```

would translate to

```

if (bar.isSome()) {
    myBar = bar
} else {
    throw NoneValueException() // Should never happen
}

```

- Any comments that appear after the last declaration in a file get *lost*:

```

class GetsTranslated {}
// class LostInTranslation {}
<EOF>

```

- It is unclear whether the comment appearing at the very top of a Java source file that has neither a package declaration nor import declarations is a header comment or relates to the declaration that follows. This makes a difference because Cangjie source files *always* have package declarations, therefore the translator must decide whether to place that comment at the very top of the output file or after the package declaration.

The translator applies the following heuristic: if that opening comment is a Javadoc-comment, it is assumed to relate to the following class/interface declaration, otherwise it is deemed to be a header comment. Consider two examples:

```

/*
 * Header comment with copyright and license
 */

class Foo {
    . . .
}

```

becomes

```

/*
 * Header comment with copyright and license
 */

package UNNAMED

class Foo {
    . . .
}

```

whereas

```

/**
 * A class that represents a foo.
 * <p>

```

```

* Implements a common foo. It must contain a bar and may link to a baz.
*/

class Foo {
    . . .
}

```

becomes

```

package UNNAMED

/**
 * A class that represents a foo.
 * <p>
 * Implements a common foo. It must contain a bar and may link to a baz.
 */
class Foo {
    . . .
}

```

7.1.2 Literals

7.1.2.1 Numeric Literals

The information about the base of numeric literals is lost in translation. All numeric literals translated from Java are decimal (base 10).

Any formatting with underscores and trailing zeroes in the fractional part is also lost.

7.1.2.2 String Literals

Unicode surrogates (`\ud800-\udfff`) in string literals are currently translated verbatim.

7.2 Types, Values, and Variables

7.2.1 Reference Types

There is no such thing as a null reference in Cangjie, so in the default “code style first” mode (`-mode=codestyle`), J2CJ assumes that all entities of Java reference types are non-nullable and emits **markers** for any explicit uses of **null**, for the developer to review and edit the resulting Cangjie code appropriately after translation:

```

class Foo {
    void baz(Foo bar) {
        Foo foo = null;
        if (bar != null) baz(null);
        Foo qux = new Foo();
    }
}

```

```

class Foo {
    init() { }
    func baz(bar: Foo): Unit {
        let foo = <-- null -->
        if (bar != <-- null -->) {
            baz(<-- null -->)
        }
        let qux = Foo()
    }
}

```

```
}
}
```

In the **semantics preservation** mode (`-mode=semantic`), all translated reference types get wrapped in `Option<T>`, with `None` representing null references, unless the translator can prove that a certain value is not **null**:

```
// -mode=semantic (boilerplate omitted for clarity)
open class Foo <: Equatable<Foo> & Hashable & ToString {
  init() { }
  public open func baz(bar: ?Foo): Unit {
    let foo: ?Foo = None
    if (bar.isSome()) {
      baz(None)
    }
    // The Java operator 'new T()' either creates an instance of T
    // or throws an exception, so it never evaluates to null:
    let qux = Foo()
  }
  . . .
}
```

NOTE: Under certain circumstances, J2CJ wraps reference types in `Option<T>` even in the “code style first” mode. A use of the below or similar **mapping** may lead to that:

```
mapping java.util.Iterator<E> => Iterator<?E> {
  next
}
```

Here, the return values **null** and `None` indicate that nothing is left to iterate over respectively in the Java and Cangjie versions of `Iterator.next`.

See also **Cast Expressions**.

7.2.2 Parameterized Types

In the current version of J2CJ, type parameter bounds are mostly unsupported and are translated into markers. This will improve in future releases.

7.2.3 Intersection Types

Intersection types are not supported and will not be until type bounds are supported.

7.3 Conversions and Contexts

7.3.1 Boxing Conversion

Boxing of value types is not required in Cangjie, as they can be used as type arguments directly, unlike the primitive types in Java. So in the default “code style first” mode (`-mode=codestyle`), the Java wrappers for primitive types, such as `java.lang.Integer`, are normally translated into the respective Cangjie value types, unless the applicable **mapping** mandates an optional value:

```
Integer baz = Integer.valueOf("42");
Integer qux = Integer.valueOf(42);
```

becomes

```
let baz = Int32.parse("42")
let qux = 42i32
```

In the **semantics preservation** mode (`-mode=semantic`), `Option<T>` wrapping is used as for the **reference types**.

```
public open func bar(): Unit {
    let baz: ?Int32 = Int32.parse("42")
    // The current version is not capable of understanding that
    // Integer.valueOf(int i) cannot return null:
    let qux: ?Int32 = 42i32
}
```

7.3.2 Unboxing Conversion

As **boxing conversions are not really translated**, so are the unboxing ones. If `Option<T>` wrapping was used in the **semantics preservation** mode (`-mode=semantic`), value retrieval is done using the `getOrThrow()` instance member function.

7.3.3 String Conversion

In Java, any reference type may be converted to type `String` by means of invoking the instance method `toString()` defined by the ultimate supertype of all Java reference types, the class `java.lang.Object`. That fact is part of what makes the **string conversion** possible, which in particular enables the string concatenation operator `+` to work when one of the operands is not a string.

In Cangjie, however, a type that does not implement the `ToStdString` interface may not be implementing the `toString` method, so there is no unified way to convert an arbitrary value to a string. Therefore, J2CJ translates *implicit* string conversions as follows:

In the default “code style first” mode (`-mode=codestyle`), it inserts a call of `toString()` at the point of such implicit string conversion regardless of whether the type in question implements the `ToStdString` interface. The resulting code may or may not compile; if it does not, it is up to the developer to either extend the type in question with the `ToStdString` interface, or devise some other way to resolve the problem.

In the **semantics preservation** mode (`-mode=semantic`), J2CJ inserts a call of a helper function `stringValueOf(Any)` instead. That call always compiles. If the actual type of a certain expression `e` does *not* implement the `ToStdString` interface, `stringValueOf(e)` returns a string similar to the one produced by the default Java `toString` method implementation.

7.4 Names

Although it is considered bad practice, a Java type may have a field, a member type, and a method share the same simple name (identifier). This works, because it is always possible to determine the meaning of a name either by syntax (only method names can be followed by “`()`”) or from context.

In Cangjie, however, member variables and functions of a type must have distinct names. (There are no member types in Cangjie, so a name clash with a member type resolves automatically.) Also, an instance member function and a static member function cannot have the same name.

The translator therefore appends “**Method**” to the names of instance methods and “**Static**” to the names of **static** methods if they clash with other field/method names:

```
int qux;  
void qux() {}  
static void qux(int x) {}
```

translates to

```
var qux: Int32 = 0  
public open func quxMethod(): Unit { }  
static func quxStatic(x: Int32): Unit { }
```

7.4.1 Fully Qualified Names

In Java, it is possible to use a fully qualified name to access a public member of another package even if that member was not imported. In Cangjie, that is not permitted. The current version of J2CJ yields simple names in such situations. A name clash is therefore possible that is *not* detected automatically.

7.4.2 Scoping???

7.5 Packages

7.5.1 Static Import

Cangjie does not distinguish import of **static** members, so Java **import static** declarations are translated into conventional Cangjie import declarations.

7.6 Classes

7.6.1 Class Declarations

7.6.1.1 Class Modifiers

7.6.1.1.1 sealed, non-sealed, and final Classes

The Cangjie modifier **sealed** limits inheritance and implementation to the current *package*, whereas a **sealed** Java class/interface can be inherited/implemented only by types declared in the same *file* or listed explicitly in the **permits** clause, which has no analogue in Cangjie.

sealed Java classes are therefore translated into ordinary Cangjie classes, so an exhaustive **switch** over a set of such classes is no longer exhaustive after translation to a **match**. That may trigger the respective Cangjie compilation errors. Consider adding the Cangjie **sealed** modifier to those classes manually after translation if they belong to the same package. Otherwise, you may have to add a dummy default (catch-all) **case _** to the Cangjie **match** expression to resolve the issue, even though that would make the code more fragile.

7.6.1.1.2 strictfp Classes

The `strictfp` modifier on classes and methods is skipped. All floating-point calculations are done in compliance with the IEEE-754 standard, which is the mode enforced by that modifier even if the hardware is capable of making such calculation with higher precision. Older versions of Java did not default to that behavior, but from Java 17 onwards `strictfp` is redundant.

7.6.1.2 Inner Classes and Enclosing Instances

Inner classes are supported by making the access modifiers on fields and methods referenced by access methods generated by `javac` under the hood less restrictive. However, there is no 100% confidence that all possible access method formats are covered.

7.6.2 Field Declarations

7.6.2.1 transient Fields

The `transient` modifier is not supported as it has no equivalent in Cangjie. It is skipped (a marker is generated), potentially breaking code relying on such fields having default values after deserialization.

7.6.2.2 volatile Fields

Changed in 1.7.2:

`volatile` fields are now supported using `std.sync.Atomic*` classes, but they are translated differently in the default “code style first” mode and the `semantics preservation` mode. See [Types of `volatile` Fields](#) for details.

The following operations on `volatile` fields of primitive types are not supported yet:

- Unary operators:

```
volatile int a;  
.  
.  
.  
a++;
```

- Compound assignment operators:

```
volatile int a;  
.  
.  
.  
a += 10;
```

The respective `markers` are inserted in the output code.

7.6.3 Method Declarations

7.6.3.1 Method Modifiers

7.6.3.1.1 native Methods

J2CJ does not support the `native` modifier. More precisely, it translates the declarations and invocations of `native` Java methods, but wraps the modifier in the “`Unsupported modifier`” marker.

Justification:

The **native** modifier in Java marks methods implemented in another programming language, such as C. A **native** method implementation must adhere to a C API called Java Native Interface (JNI), which enables accessing and instantiating Java objects, invoking their methods, throwing Java exceptions, and so on. However, most of that power comes at the cost of significant run-time overheads. *Direct byte buffers* are commonly used to circumvent some of those overheads when passing large amounts of raw data between Java code and native code.

In contrast, the Cangjie foreign function call feature is essentially a low-level mechanism for calling C functions and APIs directly from Cangjie code. The problem is that it lacks the ability to pass as parameters the instances of higher-level Cangjie types to which J2CJ translates their Java counterparts, most notably classes and interfaces, as well as arrays of those types. At the same time, Cangjie is capable of handling raw low-level data and exchanging it with foreign functions.

As a result, a direct translation of a Java **native** method declaration to a Cangjie foreign function declaration would be meaningless in the general case:

- If only data of primitive types and arrays thereof is passed back and forth and manipulated, the same result can be achieved in Cangjie with much less code.
- If Java reference types other than arrays of primitive types are substantially involved, a complete rewrite of the native part may be the only feasible option.

If the **native** methods in the application that you are converting are merely wrappers around a certain C library or API, consider utilizing the C language interoperability features of Cangjie to replace the invocations of those wrappers with direct calls of the said library or API functions in the resulting Cangjie code.

Otherwise, if the code of those **native** methods comprises a significant part of the application logic, whether for performance or legacy reasons, consider rewriting those methods in Cangjie.

In either case, some **native** methods may accept parameters of Java reference types and employ JNI functions to access them, create other instances of such types, use direct byte buffers, throw Java exceptions and so on. Such methods, as well as their usage points, require careful analysis and rewrite, which can only be done manually.

7.6.3.2 **synchronized** Methods

In the default “code style first” mode (`-mode=codestyle`), **synchronized** methods are not supported.

In the **semantics preservation** mode (`-mode=semantic`), **synchronized** methods are supported with the aid of the helper library.

7.6.3.2.1 **strictfp** Methods

The **strictfp** modifier on methods is skipped. All floating-point calculations are done in that mode.

7.6.4 **Instance Initializers**

Instance initializers are not supported and will not be supported due to Cangjie limitations. The current version of J2CJ translates them but wraps in markers.

7.6.5 Static Initializers

Static initializers are not supported and will not be supported due to the substantial differences between their semantics in Java and Cangjie. The current version of J2CJ translates them but wraps in markers.

7.6.6 Enum Types

The semantics of Java enum types and Cangjie enums differs to the extent that often makes translation of the former into the latter impractical. A Java enum type is essentially a special class, so it may have non-default constructors with parameters, constants defined as instances of anonymous classes, and so on. Such a complex Java enumeration cannot be reasonably translated into a Cangjie enum, which is a sum type.

In the simplest case, however, translation to Cangjie enums is possible. In the default “code style first” mode (`-mode=codestyle`), J2CJ translates the Java enum types that are just lists of named constants into Cangjie enums. See also the `--enum-functions` [command-line option](#).

Otherwise, and that includes the translation of such simple Java enum types in the [semantics preservation](#) mode (`-mode=semantic`), Java enum types are translated into regular Cangjie classes with private `init` functions and `static let`-fields representing enum constants. They are generally usable, but you may want to consider migrating the translated code over to proper Cangjie enums manually.

7.6.7 Record Classes

Record classes are not supported.

7.7 Interfaces

7.7.1 Interface Declarations

The `strictfp` modifier on interfaces is skipped. All floating-point calculations are done in that mode.

7.7.2 Field (Constant) Declarations

Java interfaces may contain fields, which are implicitly `public`, `static` and `final`. Such fields must be accompanied with initializer expressions, albeit not necessarily constant ones.

In contrast, Cangjie interfaces may not contain fields at all. As a workaround, J2CJ translates interface constants into read-only static properties:

```
interface Computer {
    int THE_ANSWER = 42;
}
```

```
interface Computer {
    static prop THE_ANSWER: Int32 {
        get() {
            42
        }
    }
}
```


Changed in 1.7.2:

Initializer expressions that are simple literals, as in the example above, are supported for all kinds of interfaces. More complex initializer expressions are *not* supported for interfaces that have bounded type parameters; a **marker** is generated:

```
interface I1 {
    public static String FIZZ = "Fizz"; // OK
}

interface I2<T> extends I1 {
    String FIZZBUZZ = FIZZ + "Buzz"; // OK
}

interface I3<T extends Comparable> {
    String FIZZBUZZ = I1.FIZZ + "Buzz"; // Marker will be generated
}
```

```
interface I1 {
    static prop FIZZ: String {
        get() { "Fizz" }
    }
}

private let I2_FIZZBUZZ = I2<Any>.FIZZBUZZ_init
interface I2<T> <: I1 {
    static prop FIZZBUZZ: String {
        get() { I2_FIZZBUZZ }
    }

    static prop FIZZBUZZ_init: String {
        get() { FIZZ + "Buzz" }
    }
}

private let I3_FIZZBUZZ = <-- Constants in interfaces with bounded type parameters
are unsupported: I3 -->.FIZZBUZZ_init
interface I3<T> where T <: Comparable<Any> {
    static prop FIZZBUZZ: String {
        get() { I3_FIZZBUZZ }
    }

    static prop FIZZBUZZ_init: String {
        get() { "Fizz" + "Buzz" }
    }
}
```

7.7.3 Method Declarations

static and **default** modifiers are recognized on interface methods. However, invoking the default implementation from an overridden method in a child class is not supported as it cannot be expressed in Cangjie in a straightforward manner.

7.7.4 Annotation Types

7.7.4.1 Predefined Annotation Types

Only a few selected predefined annotation types are supported. See *Predefined Annotation Types* for details.

7.7.5 Functional Interfaces

Functional interfaces are supported, but translated differently depending on their properties and translation mode.

In the default “code style first” mode (`-mode=codestyle`), uses of certain Java functional interface types are translated into uses of the equivalent Cangjie function type. This in turn enables the translation of Java **lambda expressions** into Cangjie lambdas.

In all other situations, functional interfaces are translated just like all other interfaces.

The annotation `@FunctionalInterface` is never preserved.

See *Function Types* for details.

7.8 Blocks and Statements

7.8.1 Labeled Statements

Java labeled statements are not supported and will not be supported as there is no reasonable way to translate them to Cangjie. The current version of J2CJ translates them into invalid code.

7.8.2 switch Statements

Multiple *SwitchLabels* in a single *SwitchBlockStatementGroup* are supported, but *fall through* is not.

WARNING: The current version of J2CJ translates each *SwitchBlockStatementGroup* not ending with a **break**, **return**, or **throw** statement as if it ended with a **break** statement, without producing a warning, so the resulting Cangjie code is incorrect (but may still compile). This is a known issue that will be fixed.

7.8.3 Basic for Statements

Java basic **for** statements have no direct equivalent in Cangjie and are therefore translated into Cangjie **while** loop expressions in the general case:

```
double sum = 0.0;
for(double x = 1.0; x > 0.0; x = x/2.0) {
    sum += x;
}
```

```
var sum = 0.0
var x = 1.0
while (x > 0.0) {
    sum += x
    x = x / 2.0
}
```

As the translations of any variable declarations from the *ForInit* part of the original **for** statement have to be inserted right *before* that **while** loop expression, the names of those variables may clash with the names of translated local variables declared *later* in the same Java method. In such cases, the translator additionally wraps the entire **while** loop expression in a **do-while** loop expression executing exactly once, so as to create a nested scope:

```
double sum = 0.0;
for(double x = 1.0; x > 0; x = x/2) {
    sum += x;
}

. . .

// The scope of this 'x' is from the point of declaration forward
// in both Java and Cangjie:
int x = 42;
```

```
var sum = 0.0
do {
    var x = 1.0
    while (x > 0.0) {
        sum += x
        x = x / 2.0
    }
} while (false)
let x = 42i32
return sum

// ← This do-while introduces a nested scope...
// ← ...for this 'x' to not clash with the 'x' below

// ← Here the nested scope ends
```

New in 1.7.2:

J2CJ now translates basic **for** statements of a certain “canonical” form into Cangjie **for-in** loop expressions iterating over a range:

```
int sum = 0;
for (int x = 1; x <= 10; x++) {
    sum += x;
}
```

```
var sum = 0i32
for (x in 1i32..=10i32) {
    sum += x
}
```

Consider the syntax of the Java basic **for** statement:

for ([*ForInit*] ; [*Expression*] ; [*ForUpdate*]) *Statement*

For J2CJ to translate a given basic **for** statement into a **for-in** loop expression over a range, that statement must satisfy all of the following conditions:

- The *ForInit* part must declare exactly one variable of a primitive signed integer type: **byte**, **short**, **int**, or **long**.

Suppose the name of that variable is *x*.

- The *Expression* must be a numeric comparison of *x* with the result of evaluation of a certain expression *E*. In other words, it must have one of the two forms: *x Op (E)* or *(E) Op x*, where *Op* is one of the numerical comparison operators *<*, *<=*, *>*, and *>=*. Parentheses around *E* need not be present if they won't alter the evaluation order.

- In the **semantics preservation** mode (`-mode=semantic`) only, the expression *E* must also be immutable. The current version of J2CJ only treats the following expressions as immutable in this context: literals, method parameters, **final** variables, and effectively **final** variables.

WARNING: In this mode, J2CJ should also check whether the variable declared in the *ForInit* part is modified anywhere within the *Statement*. However, that check is *not* yet implemented, so the current version may generate Cangjie source code that won't then compile. One possible remedy is replacing the faulty **for-in** loop expression with a **while** one manually. This problem will be addressed in a future release.

- The *ForUpdate* part must contain exactly one statement that either decrements, increments, or adds/subtracts the value of a numeric literal to/from the variable *x*. In other words, that statement must have one of the following forms:

```
++x
x++
x--
--x
x += Literal
x -= Literal
x = x + Literal
x = Literal + x
x = x - Literal
```

where *Literal* is a numeric literal.

7.8.4 Enhanced for Statements

Java enhanced **for** statements are fully supported and translated into **for-in** Cangjie expressions. An appropriate method *iterator()* is generated.

7.8.5 break Statements

Java **break** statements with labels are not supported and will not be supported as there is no reasonable way to translate them to Cangjie. The current version of J2CJ translates them into markers.

7.8.6 continue Statements

Java **continue** statements with labels are not supported and will not be supported as there is no reasonable way to translate them to Cangjie. The current version of J2CJ translates them into markers.

7.8.7 synchronized Statements

In the default “code style first” mode (`-mode=codestyle`), **synchronized** statements are not supported.

In the **semantics preservation** mode (`-mode=semantic`), synchronization on objects is supported by simulating the presence of a monitor in each object, which requires the use of a helper library. However, method arguments

of string and array types are passed by value in Cangjie, so synchronization on them is not possible and will never be possible.

7.8.8 try Statements

try-with-resources is not supported. The current version of J2CJ translates such statements into ordinary **try** statements, but many internal mappings are missing.

7.8.9 Pattern Matching

Pattern matching is not supported.

7.9 Expressions

7.9.1 Primary Expressions

7.9.1.1 this

As of the current version, J2CJ does translate Java for constructors that invoke instance methods of the object being constructed and/or pass **this** as a parameter, although the generated Cangjie code won't compile.

Rationale:

In Java constructors, invoking instance methods and passing **this** as a parameter is permitted, but considered bad practice, as the object is still being constructed. It only works because the Java specification guarantees that the instance fields of a newly created object will have been initialized with default values before its constructor is invoked. That is not the case in Cangjie, where invoking an instance method from a constructor is only permitted if the class is not **open** and the compiler can guarantee that all fields have been initialized prior to such invocation:

```
class Foo{
    var x: Int32
    init() {
        this.bar()    // Error: 'x' is not initialized yet
        x = 0
        this.bar()    // OK
    }
    func bar(): Int32 {
        return x
    }
}
```

Even though J2CJ generates explicit field initializers to replicate the Java default value semantics, most classes in real-world Java programs are not **final**, so they will be declared as **open** in J2CJ output, which will therefore fail to compile.

If you observe an error message such as “instance member function 'bar' cannot be accessed in the constructor of open class 'Foo'” or “'this' is not allowed to be accessed before all member variables are initialized” in the log of your Cangjie compiler as it tries to compile the translated sources, look for such usages of **this** in the original Java source code and refactor it accordingly.

7.9.1.2 Qualified **this**

Qualified **this** expressions of the form *TypeName*.**this**, which may appear within Java inner classes, are not supported.

7.9.2 Field Access Expressions

In Java, a field declared as static may nevertheless be accessed via an instance:

```
interface WithAField {
    String FIELD = "field";    // Implicitly public final static
    . . .
}

static int foo(WithAField i) {
    String s1 = i.FIELD;
    String t2 = getWAF().FIELD;    // getWAF() returns a WithAField
    . . .
}
```

but in Cangjie that is not possible.

If the expression that appears to the left of the dot . in a field access expression has no side effects, it is generally safe to replace it with the respective type name if the field is static. The current version of J2CJ does make that replacement, but only for interface constants and only if the said expression is a simple identifier. In all other cases, the replacement does not occur and the resulting Cangjie code does not compile, forcing a manual inspection.

In the default “code style first” mode (`-mode=codestyle`), that is, without `Option<T>` wrapping, the above Java code will translate to:

```
interface WithAField {
    static prop FIELD: String {
        get() {
            "field"
        }
    }
    . . .
}

static func foo(WithAField i) {
    let s1 = WithAField.FIELD;    // 'i' is possibly unused now
    let s2 = getWAF().FIELD;    // Won't compile
    . . .
}
```

NOTE: Cangjie interfaces may not contain fields at all, so Java interface constants are translated into read-only static properties when possible (see [Field \(Constant\) Declarations](#) for details).

7.9.3 Method Invocation Expressions

Method invocation expressions of the qualified **super** form:

TypeName . **super** . [*TypeArguments*] *Identifier*

are not supported, whether in inner classes or in interfaces.

7.9.4 Method Reference Expressions

Method reference expressions are supported with a couple of limitations:

- **Intersection types** are not supported at all, hence the type of a method reference expression must not be an intersection type.
- Method reference expressions appearing inside **instance initializers** and **static initializers** are not supported

The initializers are themselves not supported, but J2CJ nevertheless tries to translate them and wraps the result in **markers**. So what this actually means is that when doing so, the translator skips over method reference expressions and just copies them to the output file, wrapped in their own markers.

The following related features are supported:

- `java.util.concurrent.Callable` and interfaces from the `java.util.function` package are now mapped into function types in the default “code style first” mode (`-mode=codestyle`). Their default methods, such as `andThen()`, are still not supported.

The following related features are not supported, but may be supported in the future:

- Streams (`java.util.stream`)

7.9.5 Cast Expressions

In the **semantics preservation** mode (`-mode=semantic`), J2CJ wraps Java reference types in `Option<T>` so as to represent the Java `null` value in Cangjie as `None`. Such wrapping may also sometimes occur in the default “code style first” mode (`-mode=codestyle`). See **Reference Types** for details.

Should the type of the operand `x` of a Java cast expression `(T) x` be translated into such a wrapped type, that expression is translated as follows:

- In the default “code style first” mode (`-mode=codestyle`):

```
x.map { value => value as T' } ?.getOrThrow()
```

- In the **semantics preservation** mode (`-mode=semantic`):

- Casts to type `String`, array types, and primitive wrappers such as `Integer` are translated into calls of a helper member function `ValueBox<T>.cast<U>(?T) : ?U`:

```
ValueBox<T'>.cast(x)
```

- Casts to other Java reference types are translated into calls of a helper function `castOrThrow<T, U>(?T) : ?U`:

```
castOrThrow<Tx, T'>(x)
```

In all code templates above, the type of `x` is `Option<Tx>`, `T'` is the Cangjie type to which the translator has mapped the Java type `T`, and the result of a successful cast has the type `Option<T'>`. If the cast fails, a `ClassCastException` gets thrown.

7.9.6 Multiplicative Operators

7.9.6.1 Remainder Operator %

The operands of the remainder operator % may have floating-point types in Java, but not in Cangjie. Therefore:

- In the default mode (`-mode=codestyle`), expressions that compute a *floating-point* remainder get wrapped in **markers** for manual post-translation inspection.
- In the **semantics preservation** mode (`-mode=semantic`), such expressions are translated into calls of a helper function `rem()` that replicates the semantics of the Java operator % for floating-point operands:

```
import j2cjlib.utils.rem
...
r = rem(x, y)
```

NOTE: The semantics of the Java floating-point remainder operator % deviates from IEEE-754. Any Java code that requires the IEEE-754 semantics would employ the `java.lang.Math.IEEEremainder` static method instead of the remainder operator %.

7.9.7 Shift Operators

J2CJ translates the Java shift operators <<, >>, and >>> into their Cangjie counterparts with an appropriate bit mask applied to the second operand. If that operand is a literal, J2CJ applies the mask right away. Consider the following Java code:

```
int x = 0xdeadbeef;
int d = 42;
int y = x >> d;           // y == z == 0xffff7ab6f, not 0xffffffff, because
int z = x >> 42;          // only the last 5 bits of d and 42 are used
                           // as shift distance, and 42 & 0x1f == 10
```

As Cangjie shift operators use the value of the second operand as-is, J2CJ translates the declarations of variables `y` and `z` into:

```
var y: Int32 = x >> (d & 0x1f)
var z: Int32 = x >> 10
```

If you are sure that the distance never exceeds 31 (or 63 when the first operand is of type `Int64`), you may remove the mask application from the first expression manually:

```
var y: Int32 = x >> d      // I am confident that 0 <= d < 32
```

7.9.8 Relational Operators

7.9.8.1 Type Comparison Operator instanceof

In the **semantics preservation** mode (`-mode=semantic`), J2CJ normally **wraps reference types** in `Option<T>` due to the absence of `null` in Cangjie. The Java **instanceof** operator may then not be converted directly to the Cangjie operator `is`. A generic helper class `utils.ValueBox<T>` is used to facilitate type comparison.

Such wrapping may sometimes occur in the default “code style first” mode (`-mode=codestyle`), e.g. due to a **mapping**. If the Java type `T` was translated into `Option<T'>`, the expression “`x instance of T`” would translate into:


```
x.map { value => value is T' } ?? false
```

See [Reference Types](#) for details.

Type comparison for arrays is not supported, because arrays are not objects in Cangjie, they are structs. Also, Cangjie arrays are invariant, not covariant as in Java, where `B[]` is a subtype of `A[]` if `B` is a subtype of `A`, so comparison of array types makes little sense in Cangjie anyway.

7.9.9 Lambda Expressions

Java lambda expressions are partially supported, but only some of them get translated into Cangjie lambdas, and that only happens in the default “code style first” mode (`-mode=codestyle`).

As of the current version, J2CJ supports the following lambda-related features:

- Use of final / effectively final variables declared outside the lambda expression
- [Method references](#)
- Nested lambda expressions
- Declaration of local classes inside lambda bodies
- `java.util.concurrent.Callable` and interfaces from the `java.util.function` package are now mapped into function types in the default “code style first” mode (`-mode=codestyle`). Their default methods, such as `andThen()`, are still not supported.

Not supported, but may be supported in the future:

- Streams (`java.util.stream`)

Not supported with no plans to support:

- [Intersection types](#)
- Lambdas inside [instance initializers](#) and [static initializers](#), which are themselves not supported (translated but wrapped in [markers](#))

The discrepancies between the nature of lambda expressions in Java and Cangjie are too significant and numerous for the direct translation of the former to the latter to be possible in the general case. To work around those discrepancies, the translator has to convert Java lambda expressions into anonymous classes first and then translate those classes to Cangjie:

```
@FunctionalInterface
interface IntegerFunction {
    int f(int x);
}

public class Simple {
    static int apply(IntegerFunction f, int x) {
        return f.f(x);
    }
    public static void main(String[] args) {
        apply((x) -> x + 1, 0);
    }
}
```

```
// Non-essential boilerplate removed for clarity
interface IntegerFunction {
    func f(x: Int32): Int32
}

// Synthetic class representing the lambda
open class Simple_1 <: IntegerFunction {
    override public open func f(x: Int32): Int32 {
        return x + 1i32
    }
}

public open class Simple {
    static func apply(f: ?IntegerFunction, x: Int32): Int32 {
        return f.getOrThrow().f(x)
    }
    public static func javaMain(args: ?Array<?String>): Unit {
        apply(Simple_1(), 0i32)
    }
}
```

In the default “code style first” mode (`-mode=codestyle`), J2CJ can translate uses of certain Java functional interface types into Cangjie function types, and uses of instances of implementing classes into member function references where appropriate (see [Functional Interfaces](#) for details). That in turn enables the translation of Java lambda expressions into Cangjie lambdas.

The above example will translate to:

```
// Non-essential boilerplate removed for clarity
interface IntegerFunction {
    func f(x: Int32): Int32
}

public class Simple {
    // Notice the use of a function type instead of IntegerFunction:
    static func apply(f: (Int32) -> Int32, x: Int32): Int32 {
        return f(x)
    }
    public static func javaMain(args: Array<String>): Unit {
        // Java lambda translated into a Cangjie lambda:
        apply({ x => x + 1 }, 0i32)
    }
}
```

See [Function Types](#) for details.

7.10 Miscellaneous Non-Language Issues

7.10.1 Exception Chaining

There are no mappings for `Throwable.initCause()`, `Throwable.getCause()`, and the respective `Throwable` constructor. An attempt to instantiate a new `Throwable` object providing another exception as a `cause` is translated into a parameterless constructor invocation and a marker.

7.10.2 Metatype Programming

Only class literals and methods `Object.getClass()`, `Class.getName()`, `Class.getName()`, and `Class.getResourcesAsStream()` are supported. The Reflection API (`java.lang.reflection`) is not supported.

8 Markers

8.1 No type mapping: `<CLASS_FQ_NAME>`

Explanation: There is no mapping for the Java class with the fully qualified name `<CLASS_FQ_NAME>`.

Resolution: If there is a proper target for a **simple mapping** of `CLASS_FQ_NAME` in the Cangjie standard library or elsewhere in your Cangjie code base, try to **add such a mapping**.

Otherwise, you have two options:

- Implement a Cangjie equivalent yourself and map `CLASS_FQ_NAME` to it, or
- Refactor your Java source code so that it no longer uses that class.

It is up to you to decide which option is most effective, depending on the complexity of the class and how widespread is its use in your code base.

8.2 Missing mapping for `<CLASS_FQ_NAME>` member/constructor: `<MEMBER_NAME>`

Explanation: There is no mapping for the `<CLASS_FQ_NAME> . <MEMBER_NAME>` member or constructor with the matching number and types of parameters for the given arguments. This can mean either a complete absence of mappings for the named member or the inapplicability of the existing mappings to the given argument set.

Resolution: If there is a proper simple mapping in Cangjie, try to add it in `mappings.cjmap`. See details in the “Adding library mappings” section of README.

8.3 Missing method of mapped type `<CLASS_FQ_NAME>`: `<MEMBER_NAME>`

Explanation: The member owner `<CLASS_FQ_NAME>` is successfully mapped, but there is no proper mapping of `<MEMBER_NAME>` for the given argument set.

Resolution: If there is a proper simple mapping in Cangjie, try to add it in `mappings.cjmap`. See details in the “Adding library mappings” section of README.

8.4 Mapping is present but failed

Explanation: Mapping is present, but cannot be successfully applied due to different reasons: cannot cast arguments, cannot cast receiver, internal error etc.

Resolution: Hard to fix from the user side.

8.5 Unimplemented feature `<FEATURE_NAME>`

Explanation: The translation of the feature `<FEATURE_NAME>` is not yet implemented in J2CJ.

Resolution: Rewrite the Java source code so that it no longer uses `<FEATURE_NAME>`.

8.6 Unsupported <FEATURE_DESCRIPTION>

Explanation: The translation of the feature/corner case/construct with the <FEATURE_DESCRIPTION> description is not yet implemented in J2CJ.

Resolution: Rewrite the Java source code so that it no longer uses <FEATURE_DESCRIPTION>.

8.7 No type transform for: <EXPRESSION> <FROM_TYPE> -> <TO_TYPE>

Explanation: While translating the <EXPRESSION> J2CJ expected it to be <TO_TYPE>, but got <FROM_TYPE>.

Resolution: This marker often goes with some “No ... mapping” markers nearby. Try to resolve them first, probably this one will disappear.

8.8 Cannot infer type argument: <TYPE_PARAMETER>

Explanation: J2CJ failed to translate a generic Java type, method, or constructor to a parameterized Cangjie member, because it could not infer a type argument for the given <TYPE_PARAMETER>.

This may be a false flag caused by missing mappings, or a limitation of the current J2CJ architecture. Consider the following example:

```
class TypeParameterLostInTranslation {
    public Foo test() {
        return instantiate(Foo.class);
    }

    private <T> T instantiate(Class<T> clazz) {
        return null;
    }
}

class Foo {}
```

In Java, the type of class literals, `java.lang.Class`, is generic, whereas its Cangjie counterpart, `std.reflect.TypeInfo`, is not. And the current J2CJ architecture is not yet capable of handling this scenario well:

```
open class TypeParameterLostInTranslation <: Hashable & ToString {
    init() { }

    public open func test(): ?Foo {
        return castOrThrow<T, Foo>(instantiate<-- Cannot infer type argument: T
        -->>(TypeInfo.of<Foo>() as TypeInfo))
    }

    private func instantiate<T>(clazz: ?TypeInfo): ?T {
        return None
    }
}
```

Resolution: Most often, this error is caused by the absence of mapping for an involved Java entity and would go away if you provide such mapping. Look for “No ... mapping” markers nearby. If any are present, resolve them first and re-run J2CJ.

Otherwise, you may have hit the limitation illustrated by the above example. Editing J2CJ output manually may be the fastest way of resolution, but make sure to report the incident.

8.9 Generic instantiation failure: <CALL>

Explanation: J2CJ failed while trying to collect type arguments for the <CALL> call.

Resolution: This marker often goes with some “No ... mapping” markers nearby. Try to resolve them first, probably this one will disappear.

8.10 Constraints not satisfied <FAILURES>: <CALL>

Explanation: J2CJ failed while trying to collect type arguments for the <CALL> call.

Resolution: Hard to fix from the user side.

8.11 Expression is too deep: <EXPRESSION>

Explanation: The <EXPRESSION> has too many levels of call nesting (`a(b(c...))`) or chaining (`a().b().c()....`), which might significantly slow down the translation. The translator, guided by the expression depth limit, has cut it off to avoid the slowdown.

Resolution: In the original Java code, move inner calls outside of the <EXPRESSION> or split a chain of calls into two. For example:

- Change `a(b(c(d())))` to `var tmp = c(d()); a(b(tmp))`
- Change `a().b().c().d()` to `var tmp = a().b(); tmp.c().d()`

Another option is to raise the expression depth limit above its default value of 5 using the `max.expression.depth` J2CJ command-line option or the respective control of the IntelliJ IDEA plugin. However, keep in mind that increasing the maximum expression depth by too much may result in exponential translation time growth, up to what would look like a freeze, so use this option with caution.

8.12 Unsupported modifier: <MODIFIER>

Explanation: Either the given Java <MODIFIER> has no Cangjie counterpart, or the semantics of the latter substantially differs.

Resolution: Varies by <MODIFIER>:

- `sealed/non-sealed`: see *sealed, non-sealed, and final Classes*
- `transient`: see *transient Fields*
- `volatile`: see *volatile Fields*
- `native`: see *native Methods*

8.13 Constants in interfaces with bounded type parameters are unsupported: <INITIALIZER>

New in 1.7.2:

Explanation: Cangjie interfaces may not contain constants, so they are translated into static read-only properties. If the initializer expression is more complex than a single literal, some helper code needs to be generated. The current version of J2CJ can only generate such helper code for interfaces that have no bounded type parameters.

Resolution: Two options may be available:

1. If the initializer expression can be fully evaluated before compilation, replace it with a single literal.
2. If the initializer expression can be evaluated outside the interface, edit the generated code manually.

Consider the following example:

```
interface I<T extends Comparable> {
    String FIZZBUZZ = "Fizz" + "Buzz";
}
```

```
private let I_FIZZBUZZ = <-- Constants in interfaces with bounded type parameters
are unsupported: I3 -->.FIZZBUZZ_init
interface I<T> where T <: Comparable<Any> {
    static prop FIZZBUZZ: String {
        get() { I_FIZZBUZZ }
    }

    static prop FIZZBUZZ_init: String {
        get() { "Fizz" + "Buzz" }
    }
}
```

The first option is to replace the initializer with a literal in the Java code before translation:

```
interface I<T extends Comparable> {
    String FIZZBUZZ = "FizzBuzz";
}
```

```
interface I<T> where T <: Comparable<Any> {
    static prop FIZZBUZZ: String {
        get() { "FizzBuzz" }
    }
}
```

For the second option, it is fairly obvious that the initialization trick with the second `_init` property is not required in this case and the code can be rewritten as:

```
private let I_FIZZBUZZ = "Fizz" + "Buzz"
interface I<T> where T <: Comparable<Any> {
    static prop FIZZBUZZ: String {
        get() { I_FIZZBUZZ }
    }
}
```

If neither option applies, a more complex rewrite may be mandated.

See [Field \(Constant\) Declarations](#) for details.

8.14 Unary operators usage with volatile fields is not supported: <EXPRESSION>

New in 1.7.2:

Explanation: Unary operators and compound assignment operators on **volatile** fields are not supported yet. See **volatile Fields** for details.

Resolution: Either rewrite the original <EXPRESSION> without using unary operators or compound assignment operators on **volatile** fields and re-run J2CJ, or translate that expression manually.

8.15 Compound assignment to volatile fields is not supported: <EXPRESSION>

New in 1.7.2:

Explanation: Compound assignment operators and unary operators on **volatile** fields are not supported yet. See **volatile Fields** for details.

Resolution: Either rewrite the original <EXPRESSION> without using the compound assignment operators on a **volatile** field and re-run J2CJ, or translate that expression manually.

8.16 Internal error

Explanation: Internal error inside J2CJ. May be caused by supplying incorrect Java source code or incomplete project as input.

Resolution: Check if your Java project can be built successfully. If the build fails, fix any problems causing that fail first, then re-run J2CJ.

Otherwise, it is a genuine bug that is impossible to fix from the user side. Please submit a report, attaching the J2CJ log and a code example sufficient to reproduce the error.

9 Solutions to Common Problems

9.1 Version Compatibility

Problem: The J2CJ IDE plugin refuses to install because I am using an older version of the IDE than it requires

It is unlikely that updating everything to the earliest supported/compatible versions would render your project unbuildable or otherwise break things. To fall on the safe side, you can retrieve those versions from the respective archives instead of going straight to the latest ones, and of course make a backup first.

Start with the IDE itself:

- IntelliJ IDEA: <https://www.jetbrains.com/idea/download/other.html>
- Android Studio: <https://developer.android.com/studio/archive>

If your project uses a popular external build system such as Maven, Gradle or Ant, the updated IDE may prompt you to update the respective plugin to the latest version. Cancel that update, find out what is the earliest plugin version compatible with that specific version of IDE that you now have installed, and update the plugin manually. Then do the same for the build system.

Finally, conduct a full build, and if it is successful, **install the J2CJ plugin** and begin or resume the translation effort.

For example, suppose you are using Android Studio. Android Studio projects normally use Gradle as the build system, with the Android Gradle plugin (AGP) adding certain Android-specific features.

1. Head over to the *Android Gradle plugin and Android Studio compatibility* section of the latest Android Gradle plugin release notes at https://developer.android.com/build/releases/gradle-plugin#android_gradle_plugin_and_android_studio_compatibility and look up the minimum required AGP version in the row bearing the Android Studio version to which you have just upgraded (you may need to click **Older Versions** to see it). Update AGP to that version.
2. Update the version of the AGP in your `.gradle` file(s). Normally it is set in `build.gradle` located in the root directory of your project, in a line of the form:

```
classpath 'com.android.tools.build:gradle:3.0.0'
```

Change the version number to match the version of the AGP that you have installed on the previous step.

3. Update Gradle itself, looking up the minimum version in the *Update Gradle* section (<https://developer.android.com/build/releases/gradle-plugin#updating-gradle>)

The easiest way to update Gradle is with the `wrapper` task as described at <https://gradle.org/install/#with-the-gradle-wrapper>.